Adevar Labs

Prepared for:
Exponent

April 13, 2026

Exponent Vaults Audit Report

Version: after fix review

Table of Contents

1. Executive Summary

About Exponent

Exponent is a Solana yield trading protocol. It wraps yield-bearing assets into standardized SY tokens, splits them into principal (PT) and yield (YT) tokens, and trades them on a time-dynamic CLMM that converges at maturity. Exchange rates are computed via pluggable interfaces (Pyth, Sanctum, Hylo, Kamino, hardcoded).

Exponent Vaults is a managed vault program where managers deploy capital across DeFi strategies through a Squads multisig. It features LP governance with proposal voting, timelocks, and rejection thresholds; a sentinel role for emergency manager replacement; and a queue-fill-execute withdrawal lifecycle.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	3	3	0
Medium	13	13	0
Low	4	2	2

Enhancement opportunities: 5

Overall Security Posture

Exponent Vaults uses a defense-in-depth model: Squads multisig for manager actions, post-hook validation per integrated program, LP governance with timelocks, and a sentinel role as a final safety net. The codebase separates vault state, integration adapters, and governance flows cleanly, with typed AUM accounting via the **Number** precision library. The audit found issues primarily at the boundaries between these layers: cross-instruction state aggregation in hook validation, sign and conversion handling in AUM diffs, and ownership linkage on externally-registered positions. The Exponent team addressed nearly all findings with well-targeted fixes.

Before Fix Review Summary

The audit identified three High severity issues related to governance bypass via LP supply dilution, first-depositor vault bricking, and misinterpreted reserve threshold signs enabling full reserve depletion. Thirteen Medium severity issues were found, addressing vulnerabilities in interaction validation, AUM miscalculations, governance misdirection, and missing ownership linkages. Four Low severity issues and five enhancement opportunities were also noted, targeting protocol robustness and code quality improvements.

After Fix Review Summary

The Exponent team addressed all High and Medium severity findings. All three High severity issues and all thirteen Medium severity issues were fixed. Two of four Low severity issues were fixed and two were acknowledged. The fixes were well-implemented, with correct root-cause resolution across governance, AUM calculation, and access control domains. Residual design considerations around TWAP buffer staleness and live CLMM token composition are captured in the report assumptions. Additionally, the newly introduced pending withdrawal backlog enforcement in **validate_interaction** blocks all Squads sync transactions when withdrawals are past due, which can prevent the manager from retrieving deployed funds needed to fill those withdrawals – managers should ensure sufficient reserves ahead of withdrawal deadlines.

Launch Recommendations

I. Mandatory before launch:

- All High and Medium severity findings have been fixed. No open issues remain.

II. Strongly recommended:

- Address the acknowledged findings where feasible. Implement the remaining enhancement opportunities to improve code quality. Expand the test suite to cover edge cases identified during the audit.

III. Post-Launch Monitoring:

- Implement alerts for unusual protocol interactions, particularly around AUM changes, governance proposals, and withdrawal queue activity.
- Maintain strict operational security practices for all privileged keys – use hardware wallets for admin and multisig signers, enforce key rotation policies, limit key exposure to the minimum number of personnel, and establish incident response procedures for suspected key compromise.

Audit Scope

- **Repository:** https://github.com/exponent-finance
- **Before-Fix Review Commit Hash:** e8fdc0f8e76f33ea34b24e86e498cd854a5fd2e9
- **After-Fix Review Commit Hash:** a50f3e51a437475c6576b565e44b9243fdeb976d
- **Files/Modules in Scope:**
  - exponent/solana/programs/exponent_vaults/src/*.rs
  - exponent/solana/libraries/exponent_vaults_integrations/src/*.rs

2. Assumptions and Considerations

Audit Assumptions

Assumption 1: The sentinel role is trusted, as it has the ability to replace vault managers and intervene in vault operations during emergency scenarios.

Assumption 2: Malicious manager can inflate LP supply before proposal activation to raise the rejection threshold.

Assumption 3: The vault manager can route funds into any configured strategy or pool, including potentially malicious ones. Depositors should evaluate manager trustworthiness before committing funds.

Assumption 4: Integrated protocols (Kamino, Loopscale, CLMM, orderbook) correctly report position values and return expected token amounts, as the vault's AUM calculation relies on reading external account state that it cannot independently verify.

Assumption 5: In the event the vault is DoSed via a one-token donation, the manager is expected to recover by bundling RemoveTokenEntry + AddTokenEntry (with a fresh token account) + initial **deposit_liquidity** into a single atomic transaction.

Assumption 6: The TWAP price buffer uses a window-count bound rather than a time bound. This is only safe under the assumption that the price updater is functional and updates at a regular interval. If the updater goes offline and then resumes, stale observations remain in the buffer and continue contributing to the TWAP until new updates evict them one by one.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements–typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

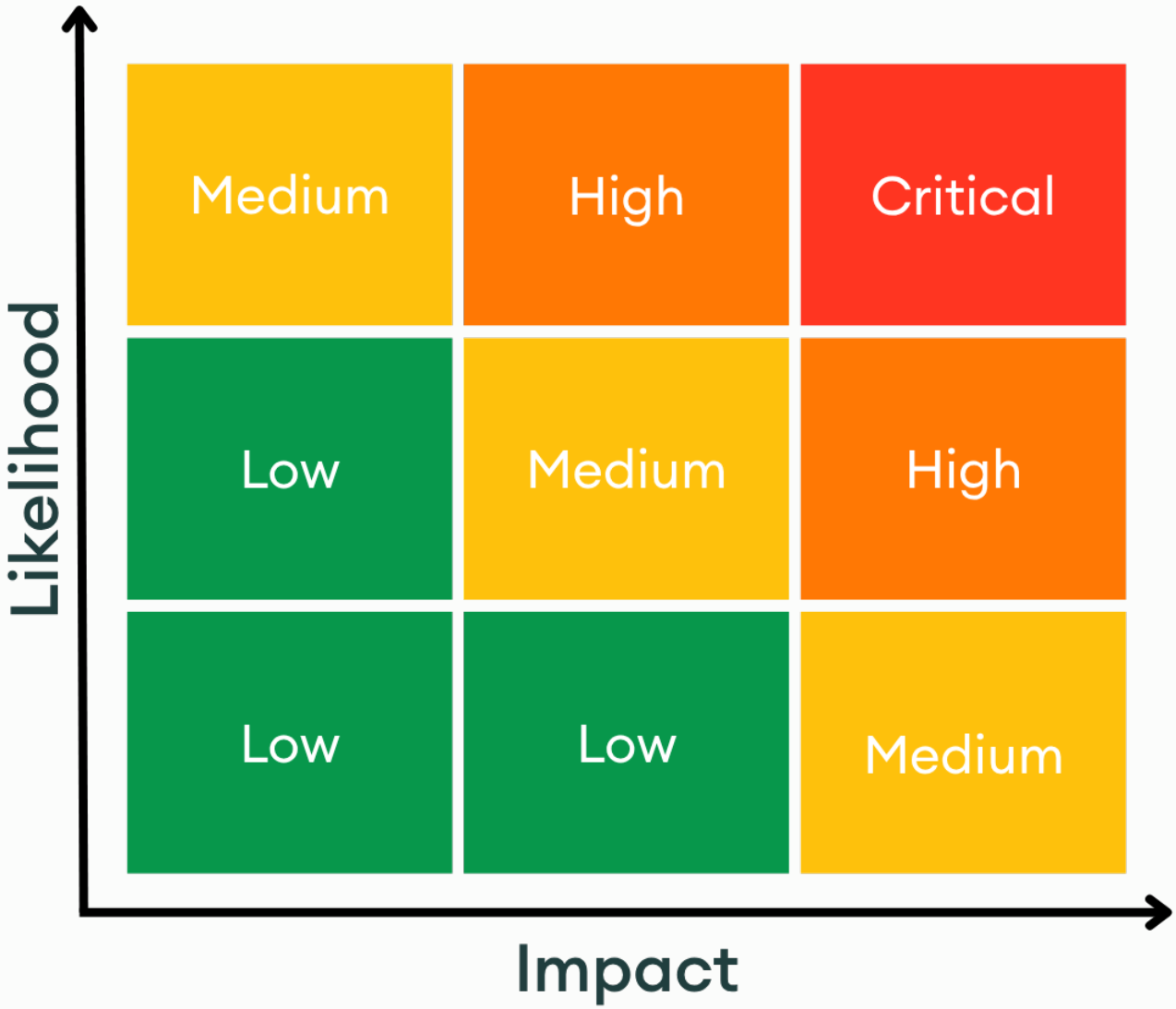
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability

- **Low**: Minimal impact or unlikely to be exploited

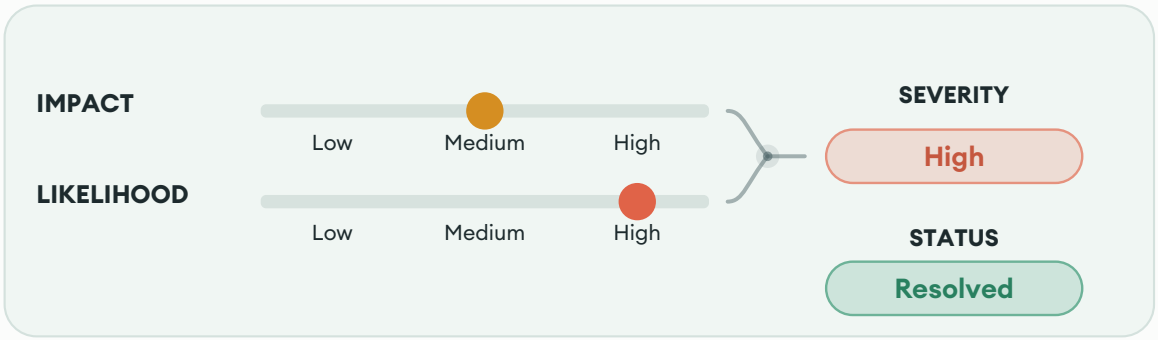
This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

H01: Misinterpreted total_aum_diff sign in reserves check will allow manager to bypass reserves threshold and deploy all vault reserves into strategy positions



LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/admin/validate_interaction.rs:L183-L193

RELEVANT SNIPPET

>_ validate_interaction.rs — RUST

```
181:             &tracked,
182:         )?;
183:         hook_validation::apply_vault_mutations(vault, &mut validation_view, &mutations
     )?;
184:     }
185: }
186:
187: // Calculate required size after validation (state may have been modified)
188: let required_size = ctx.accounts.vault.size();
189:
190: // Realloc if we need more space
191: if required_size > current_data_len {
192:     let vault_info = ctx.accounts.vault.to_account_info();
193:
194:     // Calculate additional rent needed
195:     let rent = Rent::get()?;
```

DESCRIPTION

The reserves threshold check in **validate_interaction_hook** misinterprets the sign of **total_aum_diff**. The code and its comment treat negative aum_diff as deployment from reserves into positions, but the actual convention used by every validator in the codebase is the opposite:

- **Positive aum_diff** = deployment from reserves into positions
- **mint_sy** returns **+amount_base** (standard_program.rs:135)
- Orderbook post_offer returns **+offer_aum** with comment "Posting an offer deploys AUM into a position (positive diff)" (orderbook.rs:123)
- Kamino deposit returns **+deposit_aum** (kamino.rs:323)
- **calculate_reserve_to_position_aum_diff_for_known_amount** returns **+base_i64** for **Reserve→Position** (utils.rs:237)

- **Negative aum_diff** = return from positions to reserves
- **redeem_sy** returns **-base_amount** (standard_program.rs:164)
- Kamino withdraw returns **-withdraw_aum** (kamino.rs:355)
- **calculate_reserve_to_position_aum_diff_for_known_amount** returns **-base_i64** for **Position→Reserve** (utils.rs:238)

The check:

RUST

```rust
// Reserves check: if deploying reserves into positions (negative aum_diff),
// verify remaining reserves stay above the configured threshold.
if total_aum_diff < 0 {
    let outflow = (-total_aum_diff) as u64;
    let min_reserves = vault.get_aum_in_reserves()?;
    require!(
        vault.financials.aum_in_base.saturating_sub(outflow) >= min_reserves,
        ErrorCodeVaults::ReservesExceeded
    );
}
```

Because the sign is flipped, this has two consequences:

1. Deployments go unchecked. When the manager deploys reserves (positive aum_diff), **total_aum_diff < 0** is false and the block is skipped. The manager can deploy 100% of reserves with no constraint.

2. Returns to reserves can be incorrectly blocked. When **total_aum_diff < 0** (returning from positions), the check treats the returned amount as an outflow and subtracts it from current reserves. But returns *increase* reserves, so the subtraction is backwards. For example: current reserves 3M, min reserves 2M, returning 2M from positions. The check computes **3M - 2M = 1M < 2M** and reverts, even though post-return reserves would be 5M.

RECOMMENDATION

Flip the sign condition from **< 0** to **> 0**:

```rust
if total_aum_diff > 0 {
    let outflow = total_aum_diff as u64;
    let min_reserves = vault.get_aum_in_reserves()?;
    require!(
        vault.financials.aum_in_base.saturating_sub(outflow) >= min_reserves,
        ErrorCodeVaults::ReservesExceeded
    );
}
```

FIX REVIEW NOTES

> **Developer Response:** The sign condition is now corrected in the reserves check.

> **Commit:** d060c5287fee9864f325cb7951d107191fb56de4

> **Adevar Review:** Verified. The correct sign is now used for deployment detection.

H02: One-token donation to newly initialized vault (lp_supply == 0) permanently blocks deposits with no recovery path

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/state/vault.rs:L362-L368

RELEVANT SNIPPET

>_ vault.rs — RUST

```
360:  require!(amount_base_in > 0, ErrorCodeVaults::AmountTooSmall);
361:
362:  if lp_supply == 0 {
363:      require!(
364:          self.total_aum() == 0,
365:          ErrorCodeVaults::InvalidInitialDepositState
366:      );
367:      return Ok(amount_base_in);
368:  }
369:
370:  require!(
```

DESCRIPTION

An attacker can permanently brick a newly initialized vault by sending a small amount of tokens directly to the vault's **squads_token_account** before any LP deposits occur.

When **deposit_liquidity** is called, it first recalculates AUM by reading live token balances. Then in **lp_out_from_base**, when **lp_supply == 0** it requires **total_aum() == 0**. Because the attacker's donation was picked up by **recalculate_aum_in_base**, **total_aum() > 0** and the deposit reverts.

RUST

```
if lp_supply == 0 {
      require!(
         self.total_aum() == 0,
         ErrorCodeVaults::InvalidInitialDepositState
       );
      return Ok(amount_base_in);
   }
```

All recovery paths are blocked:

1. Wrapper **manage_vault_settings** (remove the affected token entry): Blocked because **total_aum() > 0** restricts the wrapper to additions only (**LivePositionSetupOnlySupportsAdditions**).

2. Direct **manage_vault_settings**: Requires **vault.to_account_info().is_signer**. The vault PDA can only sign via **invoke_signed** from within the exponent_vaults program. Squads CPI signs with **squads_vault**, not the vault PDA, so no Squads policy can reach this path.

3. Governance (**propose_action** -> **finalize_proposal** -> **execute_proposal**): With LP supply = 0, **calculate_rejection_required(0)** returns 0. In **is_proposal_rejected**, the check **reject_votes (0) >= rejection_required (0)** evaluates to true, so every proposal is auto-rejected at finalization regardless of content.

The vault is permanently stuck: no deposits possible, no way to modify settings, no governance path available. The attacker's cost is a single token transfer of any amount (even 1 token unit).

PROOF OF CONCEPT

1. Vault is initialized with **seed_id = X**, token entry for USDC with **token_squads_account = T**.
2. Attacker transfers 1 USDC to account **T** (standard SPL transfer, no permission needed).
3. First depositor calls **deposit_liquidity**:
- **recalculate_aum_in_base** reads balance of **T** = 1 USDC, sets **total_aum() = 1**.
- **lp_supply = mint_lp.supply = 0**.
- **lp_out_from_base(0, amount)** hits **lp_supply == 0** branch.
- **require!(self.total_aum() == 0)** fails. Transaction reverts.
4. Manager attempts recovery via wrapper: **total_aum() > 0** blocks **RemoveTokenEntry**.
5. Manager attempts governance: proposal auto-rejects because **rejection_required = 0** and **0 >= 0 = true**.
6. Vault is permanently bricked.

RECOMMENDATION

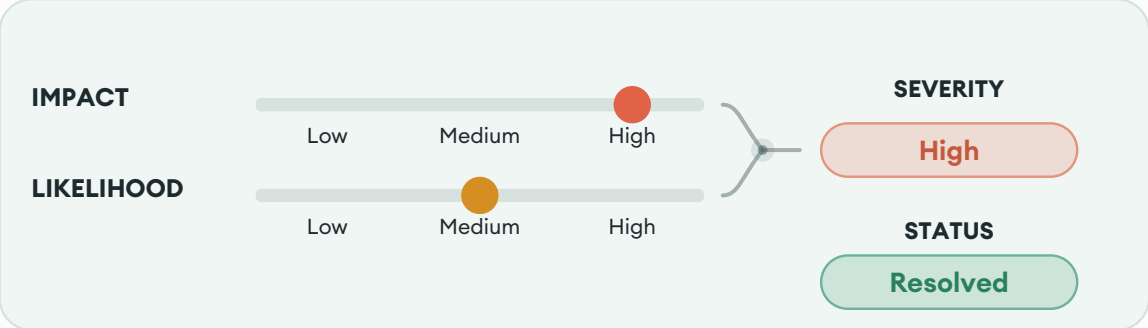
Allow the wrapper to perform removals when **lp_supply == 0** regardless of AUM, since there are no LP holders to protect.

FIX REVIEW NOTES

> **Developer Response:** The permanent brick is now recoverable through removal of the problematic **TokenEntry**. After removing the affected entry, the manager creates a new Squads token account, re-adds the **TokenEntry** with the new account, and performs the initial **deposit_liquidity**. To avoid re-exploitation between recovery and the first deposit, the manager is expected to bundle **RemoveTokenEntry** + **AddTokenEntry** + **deposit_liquidity** into a single atomic transaction.

> **Adevar Review:** Verified. The recovery path is now functional. The reliance on atomic bundling for re-initialization is captured as an assumption in the report (see Assumption 5).

H03: LP supply dilution at finalization will bypass governance rejection threshold, allowing any proposal to be forced through



LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/proposals/finalize_proposal.rs:L44

RELEVANT SNIPPET

>_ finalize_proposal.rs — RUST

```
42:  );
43:
44:  let total_lp_supply = ctx.accounts.mint_lp.supply;
45:  let rejection_required = ctx
46:      .accounts
```

DESCRIPTION

The **finalize_proposal** instruction reads the LP mint's current live supply (**mint_lp.supply**) to calculate how many reject votes are needed to block a proposal. There is no snapshot. The total supply used is whatever it happens to be at the moment **finalize_proposal** executes.

This means LP supply changes between voting end and finalization directly affect whether a proposal passes or fails. This can happen in two ways:

Malicious exploitation: A manager can intentionally inflate LP supply by calling **deposit_liquidity** after voting ends but before finalization. Since voting has closed, no new reject votes can be cast, but the existing reject votes become a smaller fraction of the inflated total supply. Both instructions can be placed in the same Solana transaction, making the attack atomic and impossible for LP holders to react to. The manager recovers their capital afterward via normal withdrawal.

Natural occurrence: Even without malicious intent, normal vault activity (new deposits from other users) between the voting period ending and someone calling **finalize_proposal** will shift the rejection threshold. A proposal that LP holders successfully rejected during voting could flip to approved simply because new deposits arrived before finalization. This makes governance outcomes non-deterministic and dependent on timing.

PROOF OF CONCEPT

Setup: Vault with 1,000,000 LP supply, **rejection_threshold_bps = 330,000** (33%). A proposal is active.

1. During voting, LP holders accumulate 350,000 reject votes (35% of supply, above the 33% threshold). The proposal should be rejected.
2. Voting period ends. No more votes can be cast.
3. Before **finalize_proposal** is called, new deposits mint ~100,000 LP (either from the manager deliberately, or from organic user deposits). Total supply becomes 1,100,000.
4. **finalize_proposal** is called. It reads **mint_lp.supply = 1,100,000**:

```
rejection_required = 1,100,000 * 330,000 / 1,000,000 = 363,000
   reject_votes = 350,000 < 363,000
```

Proposal is approved despite LP holders having voted above the threshold.

No snapshot of **total_lp_supply** is recorded at proposal creation (**propose_action.rs** does not store it). The live value at finalization time is used directly.

RECOMMENDATION

There is no simple fix for this without redesigning the governance model. The root cause is that LP supply is dynamic while the rejection-based voting model treats non-voters as implicit approvals. Any point-in-time snapshot introduces trade-offs:

Snapshot at proposal creation: Closes the post-voting dilution window, but shifts power toward LP holders. New LP minted during the voting period can be used to cast reject votes, but those votes are measured against the older, smaller snapshot supply. This gives newly minted LP outsized rejection power relative to the frozen denominator, allowing LP holders to reject any proposal by depositing and voting during the voting period.

Snapshot at voting end: The attacker simply front-runs the snapshot by depositing in the same transaction that triggers the snapshot. This does not solve the problem.

A robust solution would require snapshotting both the LP supply and per-user LP balances at a fixed point (e.g., proposal creation), and restricting voting eligibility to LP holders captured in that snapshot. This is a significant architectural change but is the standard approach in governance systems with dynamic token supply (similar to ERC-20 snapshot voting).

FIX REVIEW NOTES

> **Developer Response:** LP supply snapshot is now taken at the time of proposal activation, decoupling the rejection threshold from post-voting supply changes.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3641

> **Adevar Review:** Verified. The fix mitigates the original attack vector by preventing post-voting LP dilution from affecting finalization. The residual design limitation around pre-activation LP inflation is noted as an assumption in the report.

M01: Stale position index after Loopscale auto-removal will remove valid offer tracking from wrong orderbook when preceding orderbook has stale offers, possibly causing sudden AUM dropping

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/state/vault.rs:L580-L628

RELEVANT SNIPPET

>_ vault.rs — RUST

```rust
    pub fn size_of(force_deallocate_policy_ids_len: usize, price_id: &PriceId) -> usize {
        let force_deallocate_policy_ids_size = 4 + (force_deallocate_policy_ids_len * 8);
        32 + // mint
        price_id.size() + // price_id
        32 + // token_squads_account
        32 + // token_account_vault
        force_deallocate_policy_ids_size
    }

    pub fn size(&self) -> usize {
        let force_deallocate_policy_ids_size = 4 + (self.force_deallocate_policy_ids.len() * 8);
        32 + // mint
        self.price_id.size() + // price_id
        32 + // token_squads_account
        32 + // token_account_vault
        force_deallocate_policy_ids_size
    }
}

/// Defines how the vault deploys and manages liquidity.
#[derive(AnchorDeserialize, AnchorSerialize, Clone, Copy, Debug, PartialEq, Eq)]
pub enum VaultType {
    Orderbook,
    Generic,
}

/// Role membership lists for vault governance and operations.
#[derive(AnchorDeserialize, AnchorSerialize, Clone, Default)]
pub struct VaultRoles {
    /// Vault creator and initial admin; can create/manage other roles.
    pub manager: Vec<Pubkey>,
    /// Configures vault parameters and validation rules via policies.
```

>_ vault.rs RUST

```rust
    pub curator: Vec<Pubkey>,
    /// Executes strategy actions via adapter interface calls.
    pub allocator: Vec<Pubkey>,
    /// Oversees and controls vault risk.
    pub sentinel: Vec<Pubkey>,
}

/// Basis points multiplier for high precision (1e6)
pub const BPS_PRECISION: u64 = 1_000_000;

// ================================================================================
// Vault Status Flags
// ================================================================================

/// Flag to block liquidity withdrawals (queue_withdrawal)
pub const FLAG_WITHDRAWALS_BLOCKED: u8 = 1 << 0;
/// Flag to block manager invoke with policy
pub const FLAG_MANAGER_INVOKE_BLOCKED: u8 = 1 << 1;
/// Flag to block user invoke with policy (force_deallocate)
pub const FLAG_USER_INVOKE_BLOCKED: u8 = 1 << 2;

```

DESCRIPTION

The **OrderbookRemove** cleanup variant stores a **position_index** to identify which orderbook to clean:

RUST

```rust
// vault.rs
enum CleanupResult {
    OrderbookRemove {
        position_index: usize,       // index into strategy_positions array
        offers_to_remove: Vec<usize>, // Vec positions within offer_idx_vec
        position_aum: u64,
    },
    // ...
}
```

During **recalculate_aum_in_base**, cleanup results are collected with **position_index** from the original array ordering, then executed sequentially:

RUST

```rust
// aum.rs
let mut cleanups = Vec::with_capacity(self.strategy_positions.len());
for (position_index, position) in self.strategy_positions.iter().enumerate() {
    let (position_aum, cleanup) = position.get_aum(position_index, &ctx)?;
    // ...
    cleanups.push(cleanup);
}
// ...
for cleanup in &cleanups {
    cleanup.clean_position(self)?;
}
```

When a **LoopscaleLoanRemove** cleanup runs with **remove_position: true**, it calls **retain()**, which removes an element and shifts all subsequent indices:

```rust
// vault.rs
if *remove_position {
    vault.strategy_positions.retain(|position| {
        !matches!(position, StrategyPosition::LoopscaleLoan(entry) if entry.loan == *loan)
    });
}
```

Any subsequent **OrderbookRemove** still holds the original **position_index**, which now points to a different position. If it lands on another Orderbook, the type check passes and **offers_to_remove** is applied to the wrong orderbook:

```rust
// vault.rs
if let Some(StrategyPosition::Orderbook(entry)) =
    vault.strategy_positions.get_mut(*position_index)  // stale index -> wrong orderbook
{
    for &offer_vec_idx in offers_to_remove.iter().rev() {
        if offer_vec_idx < entry.offer_idx_vec.len() {
            entry.offer_idx_vec.remove(offer_vec_idx);  // removes valid offers
        }
    }
}
```

The cross-orderbook corruption works because **offers_to_remove** stores **Vec positions** (0, 1, 2, ...) rather than actual offer index values:

```rust
// orderbook.rs
for (offer_vec_idx, offer_idx) in entry.offer_idx_vec.iter().copied().enumerate() {
    // ...
    if user_escrow_owner != Some(*orderbook_owner) {
        offers_to_remove.push(offer_vec_idx);  // Vec position, not offer_idx value
    }
}
```

Since these are Vec positions rather than offer-specific identifiers, they are valid indices into any orderbook's **offer_idx_vec**. The more offers the wrong orderbook tracks, the more likely the Vec positions are in bounds and valid offers get removed. The removed offers are still active on the orderbook and still hold vault tokens. The vault manager can manually re-add the lost offer indices, but between the corruption and the fix, AUM drops suddenly by the value of the untracked offers.

PROOF OF CONCEPT

Vault has $10M AUM. Strategy positions: **[LoopscaleLoan(A), Orderbook(B), Orderbook(C)]**.

1. Orderbook B has two stale offers (filled, owner no longer matches) at **offer_idx_vec** Vec positions **[0, 1]**, flagged for removal during AUM calc.
2. Orderbook C has three valid offers worth $200K total at **offer_idx_vec [42, 17, 99]**.

3. Manager repays Loopscale loan A fully. The loan account is closed (0 lamports, system-owned).
4. A deposit or withdrawal triggers **recalculate_aum_in_base**:
- AUM is computed correctly ($10M), as cleanup runs after AUM summation.
- Cleanup 0: **LoopscaleLoanRemove { loan: A, remove_position: true }**. Calls **retain(...)**, removing position 0. Array shifts to **[Orderbook(B), Orderbook(C)]**.
- Cleanup 1: **OrderbookRemove { position_index: 1, offers_to_remove: [0, 1] }**. Was built for Orderbook B (originally at index 1). But **strategy_positions[1]** is now Orderbook C. Type check passes (both are Orderbook). Vec positions [0, 1] are in bounds for C (which has 3 entries), so C's valid offers at those positions are removed. C's **offer_idx_vec** becomes **[99]**.
- Cleanup 2: **OrderbookRemove { position_index: 2 }**. Index 2 is now out of bounds. **get_mut(2)** returns **None**, silently skipped. B's stale offers remain but have zero AUM value (owner doesn't match) and get cleaned up on the next cycle.
5. Next AUM recalculation: Orderbook C only iterates offer 99. Offers 42 and 17 are no longer tracked. AUM drops suddenly by ~$133K.
6. The manager can re-add the lost offer indices to restore AUM, but between the corruption and the fix, the sudden AUM drop affects LP pricing for any deposits or withdrawals in that window.

RECOMMENDATION

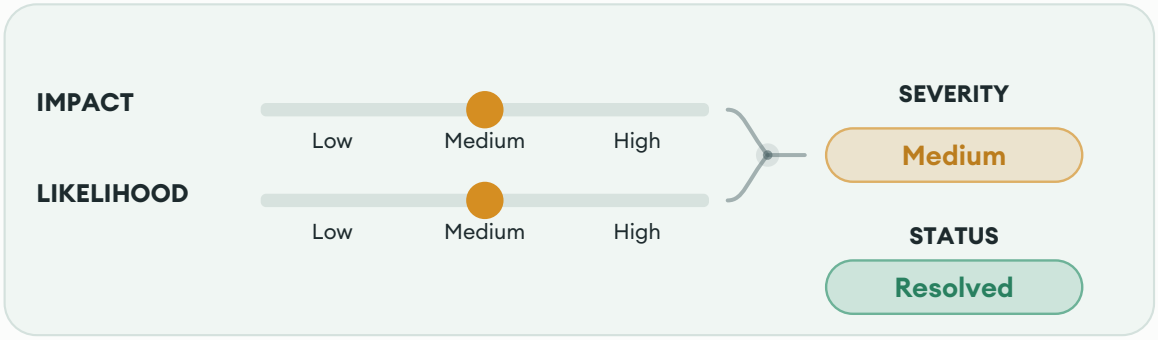
Use the orderbook's pubkey to match the target position instead of a positional index. In **CleanupResult::OrderbookRemove**, replace **position_index: usize** with **orderbook: Pubkey**, and in **clean_position**, find the matching **StrategyPosition::Orderbook(entry)** by comparing **entry.orderbook == orderbook**. This decouples cleanup targeting from array ordering, making it robust against index shifts from any current or future position removal logic.

FIX REVIEW NOTES

> **Developer Response:** **OrderbookRemove** cleanup now uses the orderbook pubkey instead of the **strategy_positions** array index. This decouples cleanup targeting from array ordering, so **retain**-triggered index shifts no longer cause cross-orderbook corruption.

> **Adevar Review:** Verified. The fix correctly resolves the root cause.

M02: Missing price conversion in standard program validate_mint_sy hook validation will miscalculate AUM change



LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/interaction_validations/standard_program.rs:L119-L133

RELEVANT SNIPPET

>_ standard_program.rs — RUST

```
117:     indices: StandardProgramAccountIndices,
118: ) -> Result<Option<Vec<VaultMutation>>> {
119:     let mint_sy_account_index = resolve_account_index(account_indexes, indices.mint_sy
     _mint_index)?;
120:     let sy_destination_account_index =
121:         resolve_account_index(account_indexes, indices.mint_sy_destination_index)?;
122:     let base_source_account_index =
123:         resolve_account_index(account_indexes, indices.mint_base_source_index)?;
124:
125:     validate_tracked_account_index(sy_destination_account_index, accounts, tracked)?;
126:     validate_tracked_account_index(base_source_account_index, accounts, tracked)?;
127:
128:     let sy_mint = *accounts[mint_sy_account_index].key;
129:     let sy_destination = *accounts[sy_destination_account_index].key;
130:     validate_tracked_account_mint(vault, &sy_destination, &sy_mint)?;
131:
132:     let amount_base = deserialize_amount(data)?;
133:     let aum_diff = i64::try_from(amount_base).map_err(|_| ErrorCodeVaults::MathOverflo
     w)?;
134:     vault.verify_aum_change(aum_diff)?;
135:
```

DESCRIPTION

validate_mint_sy computes the AUM diff directly from the raw instruction amount_base without converting it to the vault's base denomination:

RUST

```
let amount_base = deserialize_amount(data)?;
let aum_diff = i64::try_from(amount_base).map_err(|_| ErrorCodeVaults::MathOverflow)?;
Ok(Some(ValidationResult::new(aum_diff, Vec::new())))
```

The **amount_base** is in units of the standard program's base token (e.g., wSOL for a SOL-based adapter, USDC micro-units for a USDC-based adapter).

Since the vault supports multiple token entries with mints that differ from **vault.underlying_mint**, the **base_source** token can have a different denomination and price than the vault's AUM base. The hook validates that **base_source** (index **mint_base_source_index**) and **sy_destination** are tracked accounts, but never price-converts **amount_base** to the vault's AUM denomination.

Contrast with **validate_redeem_sy** in the same file (lines 139-166), which correctly price-converts via **find_tracked_account_price**:

```rust
let amount_sy = deserialize_amount(data)?;
let price_in_base = find_tracked_account_price(vault, &sy_source)?;
let base_amount = token_amount_to_base_amount(amount_sy, &price_in_base)?;
let aum_diff = -i64::try_from(base_amount).map_err(|_| ErrorCodeVaults::MathOverflow)?;
Ok(Some(ValidationResult::new(aum_diff, Vec::new())))
```

The vault can interact with any of the 5 supported standard programs (Marginfi, Kamino, Jito Restaking, Perena, Generic) and can hold tracked token accounts for any mint. When the standard program's base token differs from the vault's AUM denomination, the reserves check (performed at the top level in **validate_interaction.rs** by aggregating **total_aum_diff**) compares incompatible units. As a result, making it too permissive or too restrictive depending on the price ratio.

PROOF OF CONCEPT

1. Vault has **underlying_mint** = wSOL (9 decimals). Total AUM = 10,000 SOL. **reserves_share_bp = 2000** (20%).
- Available deployment capacity: 3,000 SOL (3x10^12)
2. Vault also tracks a USDC token account (6 decimals, price ~ 0.00667 SOL per micro-USDC).
3. Manager calls **mint_sy** on a USDC-based standard program with **amount_base = 3x10^12** (micro-USDC = $3M USDC ~ 20,000 SOL).
4. Hook returns **aum_diff = 3x10^12** (raw micro-USDC, interpreted as lamports = 3,000 SOL).
5. Top-level reserves check: treats this as 3,000 SOL deployment - passes.
6. Correct check (with conversion): 20,000 SOL deployment - should be blocked.
7. Manager deployed ~6.67x the allowed limit, draining all reserves.

RECOMMENDATION

Apply the same price conversion used by **validate_redeem_sy**. Look up the **base_source** account's price via **find_tracked_account_price** and convert

FIX REVIEW NOTES

> **Developer Response:** SY base token is now properly converted to vault base amount before AUM validation.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3654

> **Adevar Review:** Verified. The price conversion now matches the **validate_redeem_sy** pattern.

M03: Missing position ownership linkage allows allocator to register external protocol (YieldTokenPosition, KaminoObligation) positions beyond vault control

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/admin/manage_vault_settings.rs:L789-L849

RELEVANT SNIPPET

>_ manage_vault_settings.rs — RUST

```rust
787:
788: #[inline(never)]
789: fn add_yield_position_entry(
790:     vault: &mut ExponentStrategyVault,
791:     exponent_prices: &mut ExponentPrices,
792:     yield_position: Pubkey,
793:     core_vault: Pubkey,
794:     price_id_pt: PriceId,
795:     core_vault_account: &AccountInfo<'info>,
796: ) -> Result<()> {
797:     // Validate the core vault account matches the provided pubkey
798:     require_keys_eq!(
799:         *core_vault_account.key,
800:         core_vault,
801:         ErrorCodeVaults::InvalidAccountData
802:     );
803:
804:     // Deserialize the core vault to get PT mint for validation
805:     let core_vault_data = core_vault_account.try_borrow_data()?;
806:     let parsed_core_vault = ExponentCoreVault::try_deserialize(&mut &core_vault_data[..])?;
807:
808:     // Check that no yield position entry exists for this vault
809:     require!(
810:         vault.strategy_positions.iter().all(|pos| {
811:             !matches!(pos, StrategyPosition::YieldPosition(e) if e.vault == core_vault)
812:         }),
813:         ErrorCodeVaults::YieldPositionEntryAlreadyExists
814:     );
815:
816:     // Validate PT price: price_mint must match the core vault's mint_pt
817:     // This ensures the price is for the correct PT token
818:     let pt_price_ids = price_id_pt.ids()?;
```

>_ manage_vault_settings.rs RUST

```rust
    if let Some(&last_price_id) = pt_price_ids.last() {
        let pt_price_entry = exponent_prices
            .get_price(last_price_id)
            .ok_or(ErrorCodeVaults::PriceNotFound)?;
        require_keys_eq!(
            pt_price_entry.price_mint,
            parsed_core_vault.mint_pt,
            ErrorCodeVaults::InvalidPriceMint
        );
    }

    // Validate and register the PT price.
    if let Err(err) = price_id_pt.validate_price(
        exponent_prices,
        Pubkey::default(),
        None,
        vault.underlying_mint,
    ) {
        return Err(err);
    }

    vault
        .strategy_positions
        .push(StrategyPosition::YieldPosition(YieldPositionEntry {
            yield_position,
            vault: core_vault,
            price_id_pt,
        }));

    Ok(())
}

fn remove_yield_position_entry(
```

DESCRIPTION

Two position registration paths allow an allocator to register real on-chain positions from external protocols that are NOT controlled by the vault's Squads multisig (**squads_vault**). The registered position's value is counted in the vault's AUM, but the position is controlled by an external wallet that can freely deposit/withdraw without any vault signature or hook validation. This gives the allocator a bidirectional AUM manipulation primitive that completely bypasses the vault's access control.

The correct pattern exists in **add_clmm_position_entry**:

RUST

```rust
require_keys_eq!(
    lp_position.owner,
    vault.squads_vault,
    ErrorCodeVaults::InvalidAccountData
);
```

This check is missing from both affected registration paths.

RECOMMENDATION

Add ownership linkage checks to both registration paths, matching the CLMM pattern.

FIX REVIEW NOTES

> **Developer Response:** Obligation and YieldTokenPosition are now validated to belong to the vault's **squads_vault**.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3652

> **Adevar Review:** Verified. Both position types now have ownership checks matching the CLMM pattern.

M04: Manipulable CLMM token composition in vault AUM calculation will allow attackers to drain vault reserves

LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/position_values/clmm.rs:L98-L118

RELEVANT SNIPPET

>_ clmm.rs — RUST

```rust
// as remove_liquidity. Principals are kept current by swaps, so this
// reflects the real token inventory (not the theoretical AMM curve).
let (principal_pt, principal_sy) = compute_position_tokens_share_based(
    ticks,
    &lp_position.share_trackers,
    lp_position.lp_balance,
)?;

// Read-only projection of pending fees since the last snapshot update.
// This mirrors CLMM's accrue_fees_for_position logic without mutating state.
let (pending_pt, pending_sy) = compute_pending_fees_from_growth(ticks, &lp_position)?;

// Total withdrawable = principal + stored unclaimed fees + projected pending fees.
let total_pt = principal_pt
    .checked_add(lp_position.tokens_owed_pt)
    .ok_or(ErrorCodeVaults::MathOverflow)?
    .checked_add(pending_pt)
    .ok_or(ErrorCodeVaults::MathOverflow)?;
let total_sy = principal_sy
    .checked_add(lp_position.tokens_owed_sy)
    .ok_or(ErrorCodeVaults::MathOverflow)?
    .checked_add(pending_sy)
    .ok_or(ErrorCodeVaults::MathOverflow)?;

// AUM = floor(total_pt × pt_price_in_base + total_sy × sy_price_in_base)
```

DESCRIPTION

The vault computes its CLMM position AUM by reading token amounts directly from live pool state:

```rust
// clmm.rs:98-101
let (principal_pt, principal_sy) = compute_position_tokens_share_based(ticks, ...)?;

// clmm.rs:128-138
AUM = total_pt * pt_price + total_sy * sy_price
```

These principals change with every swap on the CLMM. This is the same class of vulnerability as using AMM spot reserves for price oracles: the values are live, permissionlessly manipulable, and reflect transient pool state rather than fundamental value.

When a swap moves the CLMM's spot price, the LP position rebalances between PT and SY. The vault reads the new token amounts and computes a different AUM. An attacker can capture a share of this shift by depositing into the vault, swapping on the CLMM to move the AUM, withdrawing at the shifted AUM, and swapping back.

The vault uses a fixed implied APY for PT pricing (**pt.rs:59-60**), which dampens the AUM shift when PT is present in the position. However, by pushing the spot price down to the lower tick boundary, all PT leaves the pool. With PT=0, the fixed pricing has nothing to dampen, and the full AUM shift is exposed.

The attack sequence:

1. Attacker deposits amount **D** into the vault at current AUM **A**, receiving **LP_new = D * S / A** tokens (where **S** is existing LP supply)
2. Attacker swaps SY for PT on the CLMM, pushing spot down to the lower tick boundary. The swap drains all PT from the pool (converting to SY). The vault reads the new token composition and computes a different AUM, since the fixed prices of PT and SY differ from the pool's exchange rate. The AUM shifts by **d_vault**.
3. Attacker withdraws LP at the new AUM. Their LP share is **D / (A + D)**, so they receive that fraction of the total AUM **(A + D + d_vault)**. The excess over their deposit is **D * □_vault / (A + D)**.
4. Attacker swaps back (PT for SY), restoring spot. Pays round-trip swap fees.

With a large deposit (**D >> A**), the attacker's share approaches 100% and the excess approaches **d_vault**. Net profit: **d_vault - swap_cost**.

The extraction is bounded by the vault's liquid reserves. The attacker's deposit of **D** enters the vault as base tokens, but the withdrawal of **D + share * □_vault** requires the vault to have at least **d_vault** in pre-existing reserves beyond the deposit. If the vault has deployed all capital into strategies with no idle reserves, the excess withdrawal fails.

PROOF OF CONCEPT

The following test unit demonstrate vault's AUM changes with a swap and how attacker could perform a sandwich attack to extract vault's reserve.

To reproduce:
- Place the test in: `**solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs**
- Run **cargo test -p exponent_clmm test_clmm_aum_sandwich_attack -- --nocapture 2>&1** from **solana** directory

RUST

```rust
#[test]
    fn test_clmm_aum_sandwich_attack() {
        use crate::state::market_three::helpers::add_liquidity::EffSnap;
        use crate::state::ticks::{Tick, Ticks, FarmYieldTrackers, EmissionYieldTrackers};
        use crate::utils::math::get_fee_from_amount;
        use precise_number::Number;
        use sokoban::NodeAllocatorMap;

        // ■■ Parameters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        let tau: f64 = 0.5;           // 6 months to maturity
        let r: f64 = 1.05;            // SY exchange rate
        let fixed_apy: f64 = 0.10;    // vault's fixed implied APY
        let initial_spot: f64 = 1.10;  // 10% APY, matches fixed
        let fee_rate: f64 = 1.003009;  // 30bps: 1/(1-0.003)

        let l_total: f64 = 10_000_000.0;
        let vault_share: f64 = 0.50;

        // ■■ Effective price (add_liquidity.rs:45) ■■■■■■■■■■■■■■■■■■■■■
        let snap = EffSnap {
            time_factor: tau,
            sy_exchange_rate: r,
        };

        let c_initial = snap.get_effective_price(initial_spot);
        let c_upper = snap.get_effective_price(1.20_f64);
        let c_lower = snap.get_effective_price(1.00_f64);

        // ■■ Initial principals ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        let init_principal_pt = (l_total * (initial_spot - 1.00)).floor() as u64;
        let init_principal_sy = (l_total * (c_initial - c_upper)).floor() as u64;
        let total_shares = 1_000_000u64;
        let vault_shares = (total_shares as f64 * vault_share) as u64;

        // ■■ Set up Ticks ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        let mut ticks: Ticks = unsafe { std::mem::zeroed() };
        ticks.market = anchor_lang::prelude::Pubkey::default();
        ticks.ticks_tree.initialize();
        ticks.current_spot_price = initial_spot;

        let mut lower_tick: Tick = unsafe { std::mem::zeroed() };
        lower_tick.spot_price = 1.00;
        lower_tick.principal_pt = init_principal_pt;
        lower_tick.principal_sy = init_principal_sy;
        lower_tick.principal_share_supply = Number::from_natural_u64(total_shares);
        lower_tick.farms = FarmYieldTrackers::default();
        lower_tick.emissions = EmissionYieldTrackers::default();
        let lower_idx = ticks.ticks_tree.insert(0u32, lower_tick).unwrap();

        let mut upper_tick: Tick = unsafe { std::mem::zeroed() };
        upper_tick.spot_price = 1.20;
        upper_tick.farms = FarmYieldTrackers::default();
        upper_tick.emissions = EmissionYieldTrackers::default();
```

... continued RUST

```rust
        let _upper_idx = ticks.ticks_tree.insert(200_000u32, upper_tick).unwrap();

        ticks.current_tick = lower_idx;
        ticks.current_prefix_sum = l_total as u64;

        // ■■ Share-based token read (clmm.rs:290-333) ■■■■■■■■■■■■■■■■■
        let compute_vault_tokens = |ticks: &Ticks, idx: u32, shares: u64| -> (u64, u64) {
            let tick = &ticks.ticks_tree.get_node(idx).value;
            let supply = tick.principal_share_supply.fast_floor_u128();
            let burn = shares as u128;
            let pt = if supply > 0 { (burn * tick.principal_pt as u128 / supply) as u64 } else
    { 0 };
            let sy = if supply > 0 { (burn * tick.principal_sy as u128 / supply) as u64 } else
    { 0 };
            (pt, sy)
        };

        // ■■ Pricing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        let vault_pt_price = 1.0 / (1.0 + fixed_apy * tau); // pt.rs:60
        let sy_price = r;

        let vault_aum = |pt: u64, sy: u64| -> f64 {
            pt as f64 * vault_pt_price + sy as f64 * sy_price
        };

        // ■■ STEP 1: Initial AUM ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        let (v_pt_init, v_sy_init) = compute_vault_tokens(&ticks, lower_idx, vault_shares);
        let aum_init = vault_aum(v_pt_init, v_sy_init);

        println!("\n======== PoC: CLMM AUM Manipulation ========");
        println!("Setup: tau={}, r={}, fixed_apy={:.0}%, L={}M, vault={}%",
            tau, r, fixed_apy * 100.0, l_total / 1e6, vault_share * 100.0);
        println!("Tick range: [1.00, 1.20], initial spot=1.10");
        println!("Initial: PT={}, SY={}, AUM={:.0}\n", v_pt_init, v_sy_init, aum_init);

        // ■■ STEP 2: Push spot DOWN to boundary (SY->PT swap, PT=0) ■■
        // Swap math from swap.rs:420-429 (SyToPt direction):
        //   principal_pt -= pt_out_gross
        //   principal_sy += sy_in
        let target_spot = 1.00_f64; // lower boundary
        let c_target = snap.get_effective_price(target_spot);
        let pt_out_gross = (l_total * (initial_spot - target_spot)).floor() as u64;
        let sy_in = (l_total * (c_target - c_initial)).ceil() as u64;
        let pt_fee = get_fee_from_amount(pt_out_gross, fee_rate);

        // Apply swap state update (swap.rs:420-429)
        {
            let tick = &mut ticks.ticks_tree.get_node_mut(lower_idx).value;
            tick.principal_pt = tick.principal_pt.checked_sub(pt_out_gross).unwrap();
            tick.principal_sy = tick.principal_sy.checked_add(sy_in).unwrap();
        }
        ticks.current_spot_price = target_spot;
```

... continued RUST

```rust
        // ■■ STEP 3: Post-swap AUM ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        let (v_pt_post, v_sy_post) = compute_vault_tokens(&ticks, lower_idx, vault_shares);
        let aum_post = vault_aum(v_pt_post, v_sy_post);
        let delta_vault = aum_post - aum_init;

        println!("After SY->PT swap (spot 1.10 -> 1.00, all PT exits):");
        println!("  PT={}, SY={}, AUM={:.0}", v_pt_post, v_sy_post, aum_post);
        println!("  d_vault = {:+.0} (AUM shift from composition change)", delta_vault);

        // PT should be zero at the boundary
        assert_eq!(v_pt_post, 0, "PT must be 0 at lower boundary");
        // AUM must have increased (composition shift raises vault's valuation)
        assert!(delta_vault > 10_000.0,
            "d_vault={:.0} should be >10k when pushing to boundary", delta_vault);

        // ■■ STEP 4: Correct profit formula ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        // profit = d_vault - swap_cost  (with flash loan, attacker share -> 100%)
        //
        // Round-trip swap cost:
        //   Forward (SY->PT): fee on PT output
        //   Reverse (PT->SY): fee on SY output
        let c_delta = (c_target - c_initial).abs();
        let spot_delta = (initial_spot - target_spot).abs();
        let fwd_cost = pt_fee as f64 * vault_pt_price;
        let rev_sy_out = (l_total * c_delta).floor() as u64;
        let rev_fee = get_fee_from_amount(rev_sy_out, fee_rate);
        let rev_cost = rev_fee as f64 * sy_price;
        let swap_cost = fwd_cost + rev_cost;
        let net_profit = delta_vault - swap_cost;

        println!("\nAttack (deposit-swap-withdraw with flash loan):");
        println!("  d_vault:    {:+.0}", delta_vault);
        println!("  swap cost:  {:.0} (fwd={:.0}, rev={:.0})", swap_cost, fwd_cost, rev_cost);
        println!("  NET PROFIT: {:+.0}", net_profit);

        assert!(net_profit > 0.0,
            "Attack must be profitable: d_vault={:.0}, cost={:.0}, net={:.0}",
            delta_vault, swap_cost, net_profit);

        // ■■ STEP 5: Show masking effect ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        // Compare: pushing UP to 1.20 (all PT, maximum masking)
        // Reset tick state
        {
            let tick = &mut ticks.ticks_tree.get_node_mut(lower_idx).value;
            tick.principal_pt = init_principal_pt;
            tick.principal_sy = init_principal_sy;
        }
        ticks.current_spot_price = initial_spot;

        // Push UP: PT->SY swap (swap.rs:847-858)
        let c_up = snap.get_effective_price(1.20);
        let pt_in_up = (l_total * (1.20 - initial_spot)).floor() as u64;
        let sy_out_up = (l_total * (c_initial - c_up)).floor() as u64;
```

... continued RUST

```
        {
            let tick = &mut ticks.ticks_tree.get_node_mut(lower_idx).value;
            tick.principal_pt = tick.principal_pt.checked_add(pt_in_up).unwrap();
            tick.principal_sy = tick.principal_sy.checked_sub(sy_out_up).unwrap();
        }

        let (vp_up, vs_up) = compute_vault_tokens(&ticks, lower_idx, vault_shares);
        let aum_up = vault_aum(vp_up, vs_up);
        let delta_up = aum_up - aum_init;

        let sy_fee_up = get_fee_from_amount(sy_out_up, fee_rate);
        let fwd_up = sy_fee_up as f64 * sy_price;
        let rev_pt_up = (l_total * (1.20 - initial_spot)).floor() as u64;
        let rev_fee_up = get_fee_from_amount(rev_pt_up, fee_rate);
        let rev_up = rev_fee_up as f64 * vault_pt_price;
        let cost_up = fwd_up + rev_up;

        println!("\nComparison (masking effect):");
        println!("  DOWN to 1.00 (PT=0, no masking): d_vault={:+.0}, cost={:.0}, net={:+.0}",
            delta_vault, swap_cost, net_profit);
        println!("  UP to 1.20 (all PT, max masking): d_vault={:+.0}, cost={:.0}, net={:+.0}",
            delta_up, cost_up, delta_up - cost_up);
        println!("  Masking absorbs {:.0} of AUM change in UP direction", delta_vault - delta_
up);

        assert!(delta_vault > delta_up,
            "DOWN (unmasked) must give larger d_vault than UP (masked)");

        println!("\n================================================\n");
    }
```

RECOMMENDATION

The vault's normal withdrawal path (queued, filled by manager) already breaks atomic execution and is not vulnerable. The fast withdrawal path is the vulnerable surface. Alternatively, enforce a minimum holding period between deposit and fast withdrawal, or use a prior-slot AUM snapshot for fast withdrawal pricing.

FIX REVIEW NOTES

> **Developer Response:** Implemented a windowed TWAP with configurable window for implied APY pricing of PT/YT, using a separate price account that stores interface accounts and TWAP entries.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3659

> **Adevar Review:** The windowed TWAP approach smooths implied APY and is a significant improvement over the previous all-time fixed pricing. The implementation uses a window-count bound rather than a time bound for the price buffer, which is safe under the assumption that the price updater operates on a regular interval (see Assumption 6). Additionally, the LP position value still derives token composition from live pool state – the TWAP only smooths prices, not quantities – so the manipulation surface is reduced but not fully eliminated.

M05: wrapper_merge hook does not validate base token destination, allowing policy executor to route vault funds to an external account

LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/interaction_validations/utils.rs:L85-L89

RELEVANT SNIPPET

>_ utils.rs — RUST

```
83: }
84:
85: pub(super) fn validate_tracked_account_index(
86:     account_index: usize,
87:     accounts: &[AccountInfo],
88:     tracked: &BTreeSet<Pubkey>,
89: ) -> Result<()> {
90:     require!(
91:         account_index < accounts.len() && tracked.contains(accounts[account_index].key
      ),
```

DESCRIPTION

wrapper_merge converts PT + YT into SY, then redeems SY to base tokens in a single instruction. The hook validation checks that the PT and YT source accounts are tracked:

RUST

```
validate_tracked_account_index(yt_index, accounts, tracked)?;
validate_tracked_account_index(pt_index, accounts, tracked)?;
```

But the base token destination is not validated. The wrapper_merge instruction internally calls cpi_redeem_sy which converts SY to base and sends it to a destination specified in redeem_sy_rem_accounts. The hook has access to the redeem accounts via remaining_accounts and the instruction data contains redeem_sy_accounts_until which marks their offset, but the validation does not inspect them.

The policy executor controls where the base output goes via redeem_sy_rem_accounts. Since the destination is not validated, the policy executor can route the base tokens to any account, including a personal account outside the vault's control. PT+YT are burned from the vault's tracked positions, but the base output goes directly to the executor.

PROOF OF CONCEPT

1. Vault holds 200 PT and 200 YT in tracked positions.
2. Policy executor calls **wrapper_merge(amount_py=200)** via Squads, setting **redeem_sy_rem_accounts** so the base destination is an external account.
3. Hook validates PT+YT sources are tracked, computes negative **aum_diff**. Passes.
4. PT+YT burned, SY minted to intermediate account, SY redeemed to base at untracked destination.
5. AUM drops by PT+YT value. Base tokens are outside tracked accounts.

RECOMMENDATION

Validate the base token destination from the redeem step. The redeem accounts are in **remaining_accounts** at a known offset (**redeem_sy_accounts_until**). The hook can use the standard program's account layout indices to identify the base destination account and check it against the tracked set.

FIX REVIEW NOTES

> **Developer Response:** Base token destination account is now tracked and validated to belong to a vault token entry. Additionally, a check was added to ensure the generic standard program does not have a hook.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3649

> **Adevar Review:** Verified. Token destination is checked via **resolve_account_index**.

M06: wrapper_post_offer hook passes pre-conversion base amount to AUM calculation, overstating position deployment

LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/interaction_validations/orderbook.rs:L338-L341

RELEVANT SNIPPET

>_ orderbook.rs — RUST

```
336:
337: // Calculate AUM impact
338: let params = PostOfferParams {
339:     amount: args._amount_base,
340:     offer_type: args.offer_type,
341:     virtual_offer: args._virtual_offer,
342: };
343: let orderbook_account = &accounts[context.orderbook_account_index];
```

DESCRIPTION

calculate_offer_aum_in_base computes the AUM impact of a posted offer by converting the offer amount to SY equivalent (via **get_sy_in_offer**), then multiplying by **price_sy** to get the base value. This assumes the input amount is in the offer's native token denomination.

This assumption holds for the raw **post_offer** validation path. But **validate_wrapper_post_offer** passes **args._amount_base** from the wrapper instruction data, which is in underlying/base denomination before the wrapper's internal SY minting step:

RUST

```
// Hook validation (interaction_validations/orderbook.rs):
let params = PostOfferParams {
    amount: args._amount_base,  // pre-conversion base amount
    ...
};
let offer_aum = calculate_offer_aum_in_base(vault, context.entry, &params, orderbook_account)?
  ;
```

The **wrapper_post_offer** instruction (exponent_orderbook) performs base-to-SY minting internally for non-virtual BuyYt and virtual SellYt offers:

```rust
// Wrapper handler (exponent_orderbook):
let amount = if mint_sy_yt_accounts_until != 0 && (...) {
    sy_cpi::cpi_mint_sy(amount_base, ...).sy_out_amount  // SY amount < amount_base
} else {
    amount_base
};
// Offer is posted with `amount` (SY), not `amount_base` (underlying)
```

The hook doesn't replicate this conversion. For non-virtual BuyYt, **get_sy_in_offer** returns the input directly (treating it as SY), then **calculate_offer_aum_in_base** multiplies by **price_sy**.

```rust
pub fn get_sy_in_offer(offer: &Offer, sy_exchange_rate: f64, pt_price_in_quote_asset: f64) ->
  u64 {
    let offer_type = offer.get_type();

    if offer.is_virtual() {
        if offer_type == OfferType::BuyYt {
            let pt_to_sy_rate = pt_price_in_quote_asset / sy_exchange_rate;
            ((offer.amount as f64) * pt_to_sy_rate).floor() as u64
        } else {
            offer.amount
        }
    } else if offer_type == OfferType::BuyYt {
        offer.amount // <-- non-virtual BuyYt just returns passing amount from offer
    } else {
        let yt_to_sy_rate = (1.0 - pt_price_in_quote_asset) / sy_exchange_rate;
        ((offer.amount as f64) * yt_to_sy_rate).floor() as u64
    }
}
```

But the input is already in base units, so **price_sy** gets applied to a base-denominated value, causing an overstatement of the position deployment.

PROOF OF CONCEPT

1. Vault has orderbook entry. SY exchange rate = 1.10 (10% yield accrued).
2. **effective_free_aum = 10000**, **aum_in_positions = 9500** (500 headroom).
3. Manager submits **wrapper_post_offer** with **amount_base = 500**, non-virtual BuyYt, **mint_sy_yt_accounts_until > 0**.
4. Wrapper mints SY: **sy_out = 500 / 1.10 = 454 SY**. Offer posted with 454 SY.
5. Hook reads **_amount_base = 500**, treats as SY. Computes: **500 * 1.10 = 550**.
6. Check: **9500 + 550 = 10050 > 10000**. Reverts with **ReservesExceeded**.
7. Real deployment: **454 * 1.10 = 500**. Real check: **9500 + 500 = 10000 <= 10000**. Should pass.

RECOMMENDATION

In **validate_wrapper_post_offer**, read **last_sy_exchange_rate** from the orderbook account and convert **_amount_base** to SY (by dividing by the rate) before passing to **calculate_offer_aum_in_base**. This conversion should only apply for offer types where the wrapper mints SY internally: non-virtual BuyYt and virtual SellYt. For other offer types, **_amount_base** is already in native denomination and should be passed as-is.

RUST

```rust
let amount_for_aum = if !args._virtual_offer && args.offer_type == OfferType::BuyYt
    || args._virtual_offer && args.offer_type == OfferType::SellYt
{
    (args._amount_base as f64 / sy_exchange_rate).floor() as u64
} else {
    args._amount_base
};
```

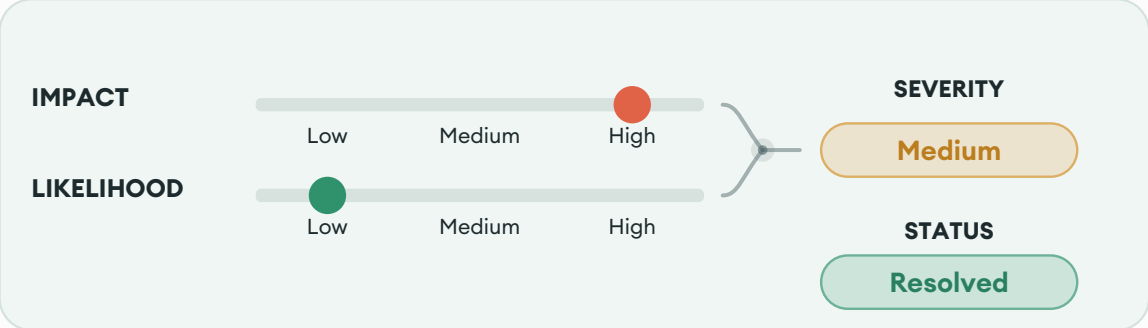
FIX REVIEW NOTES

> **Developer Response:** The amount is now properly recalculated before passing to **calculate_offer_aum_in_base**.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3648

> **Adevar Review:** Verified. The base-to-SY conversion is now applied before AUM calculation.

M07: Caller-supplied lending_program_id allows registration of fake Kamino obligation with arbitrary collateral data



LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/wrappers/wrapper_manager_update_position.rs:L58-L62

RELEVANT SNIPPET

>_ wrapper_manager_update_position.rs — RUST

```
56:     .ok_or(ErrorCodeVaults::MissingAccount)?;
57:
58: require_keys_eq!(
59:     *obligation_info.owner,
60:     entry.lending_program_id,
61:     ErrorCodeVaults::InvalidAccountData
62: );
63:
64: let obligation_data = obligation_info.try_borrow_data()?;
```

DESCRIPTION

The KaminoObligationEntry stores a lending_program_id field that is entirely caller-supplied. When a Kamino obligation is registered as a vault strategy position, the owner of the obligation account is validated against this user-supplied value rather than a hardcoded known Kamino lending program ID. The same circular check is used during AUM recalculation.

RUST

```
require_keys_eq!(
        *obligation_info.owner,
        entry.lending_program_id, // <-- caller-supplied
        ErrorCodeVaults::InvalidAccountData
    );
```

This means a malicious allocator (semi-trusted role that can add strategy positions to a live vault) can:

1. Deploy their own custom Solana program
2. Create a fake obligation account under that program, with inflated market_value_sf in the collateral deposits and zero borrows
3. Call wrapper_manager_update_position with AddKaminoObligationEntry, setting lending_program_id to their custom program

All existing guards are bypassed:

- **Owner check** (**wrapper_manager_update_position.rs:59-63**): Validates **obligation_info.owner == entry.lending_program_id**, but the caller set **lending_program_id** to their own program. Compare with the correct pattern in **validate_orderbook_accounts** which uses the hardcoded **exponent_core::ID**.
- **Discriminator check**: The 8-byte discriminator is a known constant. Any program can write these bytes.
- **Staleness check** (**kamino.rs:153-161**): The **last_update** field is inside the caller-controlled data. The **min_price_status_flags** is also caller-supplied.
- **Price validation** (**manager_update_position.rs:339-346**): Validates that price entries exist in ExponentPrices, but this constrains the **price** not the **amount**. The caller controls **market_value_sf** (amount) in the fake obligation data.

Once the fake obligation is registered, every subsequent **recalculate_aum_in_base** call reads the inflated collateral values, inflating the vault's total AUM. The malicious allocator can then withdraw LP at the inflated rate, extracting tokens belonging to other LP holders.

RECOMMENDATION

Validate the obligation account owner against a hardcoded set of known Kamino lending program IDs, rather than accepting it as a caller-supplied parameter

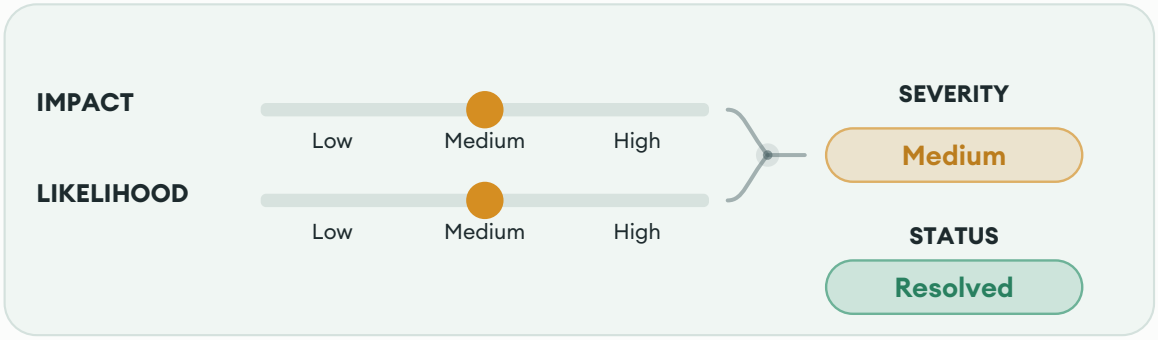
FIX REVIEW NOTES

> **Developer Response:** Obligation owner is now enforced to equal the Kamino program ID.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3647

> **Adevar Review:** Verified. The circular validation is eliminated by hardcoding the expected owner.

M08: Withdrawal Queue DoS: Users Can Block Manager Operations by Cycling Queue/Cancel Withdrawals



LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/validation_view.rs:L18-L30

RELEVANT SNIPPET

>_ validation_view.rs — RUST

```rust
impl VaultValidationView<'_> {
    pub fn verify_aum_change(&self, aum_diff: i64) -> Result<()> {
        let effective_free_aum = self.effective_free_aum_in_base as i128;
        let current_in_positions = self.aum_in_positions_in_base as i128;
        let new_in_positions = current_in_positions
            .checked_add(aum_diff as i128)
            .ok_or(ErrorCodeVaults::MathOverflow)?;

        require!(
            new_in_positions <= effective_free_aum,
            ErrorCodeVaults::ReservesExceeded
        );
        Ok(())
    }

    pub fn apply_mutation(&mut self, mutation: &VaultMutation) -> Result<()> {
```

DESCRIPTION

An LP holder can prevent the vault manager from deploying capital into any strategy by repeatedly queuing and cancelling withdrawals at zero cost.

queue_withdrawal increments **vault.financials.lp_locked_for_withdrawal** by the user's LP amount:

RUST

```rust
// queue_withdrawal.rs:129-135
ctx.accounts.vault.financials.lp_locked_for_withdrawal = ctx
    .accounts.vault.financials.lp_locked_for_withdrawal
    .checked_add(lp_amount)
    .ok_or(ErrorCodeVaults::MathOverflow)?;
```

When the manager executes any interaction through Squads, the post-hook builds a **VaultValidationView** that subtracts the locked amount from free AUM:

```rust
// hook_validation.rs:122-125
let effective_free_aum_in_base = vault
    .get_free_aum()?
    .checked_sub(aum_locked_for_withdrawals)
    .ok_or(ErrorCodeVaults::MathOverflow)?;
```

Every subsequent **verify_aum_change** call checks **new_in_positions <= effective_free_aum**. With a large **aum_locked_for_withdrawals**, **effective_free_aum** shrinks.

All manager operations that deploy capital (Kamino deposits, orderbook offers, token transfers) fail.

The user then calls **cancel_withdrawal**, which requires only that **fills** is empty (no partial fills have occurred). Cancellation returns LP tokens to the user at zero cost and decrements **lp_locked_for_withdrawal**:

```rust
// cancel_withdrawal.rs:86-105
require!(ctx.accounts.withdrawal_account.fills.is_empty(), ...);
ctx.accounts.vault.financials.lp_locked_for_withdrawal = ctx
    .accounts.vault.financials.lp_locked_for_withdrawal
    .checked_sub(lp_remaining)
    .ok_or(ErrorCodeVaults::MathOverflow)?;
```

The user can repeat this cycle indefinitely. A single whale LP holder, or a coordinated group, can completely block vault operations for as long as they choose. Users earn no yield while capital sits idle, causing indirect economic damage to all depositors.

RECOMMENDATION

Introduce a cancellation cooldown or a small fee on withdrawal cancellations to make the attack economically costly.

For example, require that a withdrawal request exists for a minimum number of slots before it can be cancelled:

FIX REVIEW NOTES

> **Developer Response:** Added a cancellation cooldown period and a window-based limit for instant withdrawals. The manager is now required to process queued withdrawals via **pending_withdrawal_backlog_due_at**.

> **Adevar Review:** The cancellation cooldown addresses the zero-cost cycling attack.

M09: Governance policy-add proposals can be misdirected to attacker-controlled Squads settings, silently sabotaging approved proposals

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/internal/action_execution.rs:L195

RELEVANT SNIPPET

>_ action_execution.rs — RUST

```
193: );
194:
195: let squads_settings = &remaining_accounts[0];
196: let squads_settings_transaction = &remaining_accounts[1];
197: let squads_proposal = &remaining_accounts[2];
```

DESCRIPTION

When a governance **PolicyAction::Add** proposal is executed, the Squads settings account is taken from caller-supplied **remaining_accounts[0]** via **split_policy_accounts** (action_execution.rs:195). This function only validates that **remaining_accounts[4]** is the Squads program ID. It does NOT validate that **remaining_accounts[0]** matches **vault.squads_settings**.

In contrast, the direct **AddPolicy** instruction has **has_one = squads_settings** (manage_policy.rs:22), which enforces the match. The governance path bypasses this constraint.

An attacker can exploit this by:

1. Creating their own Squads smart account via **create_smart_account** with the target vault PDA listed as a signer. The Squads program accepts any pubkey as a signer without consent -- the vault PDA does not need to approve this.

2. When a **PolicyAction::Add** proposal is approved and the timelock expires, the attacker calls **execute_proposal** with **remaining_accounts[0]** pointing to their Squads settings instead of the vault's.

3. Inside **apply_add_policy**, all operations succeed:
- **load_squads_settings(attacker_settings)** passes -- valid Squads account format
- **validate_new_policy_account(policy, attacker_settings.key(), seed)** passes -- attacker provides the matching PDA derived from their settings
- The Squads CPIs (create settings transaction, create proposal, approve, execute) succeed because the vault PDA is a signer on the attacker's settings and signs via **invoke_signed** with **vault.signer_seeds()**

4. The policy is created on the attacker's Squads settings, not on **vault.squads_settings**. The proposal is marked **Executed**. The vault's actual configuration is unchanged.

PolicyAction::Update and **PolicyAction::Remove** are not affected because **validate_policy_account** (action_execution.rs:107, 125) derives the expected policy PDA from **vault.squads_settings**, pinning those operations to the correct settings account.

The direct **AddPolicy** instruction enforces **PolicyConfig::validate()**, which restricts policy creation to **ProgramInteraction** type with **threshold > 0** and non-empty **instructions_constraints**. Certain policy types can only be added through governance: **SpendingLimit**, **SettingsChange**, **InternalFundTransfer**, zero-threshold policies, and unconstrained interaction policies. The manager has no other path to create these.

This means an attacker can repeatedly block any governance-only policy type from being added. Each blocked attempt forces a new governance cycle (voting period + timelock, default 72 hours). The attacker's one-time setup cost (creating their Squads settings) is already done, and they can misdirect every subsequent attempt. Since the proposal is marked **Executed** on success, the sabotage is not immediately visible -- the proposer would need to independently verify that the policy appeared on the vault's actual Squads settings to detect the misdirection.

PROOF OF CONCEPT

1. Vault has **squads_settings = SettingsA**. LP holders approve a **PolicyAction::Add** proposal to add a protective validation hook policy.

2. Attacker calls Squads **create_smart_account**:

```rust
signers: [{ key: vault_PDA, permissions: 0b111 }]
   threshold: 1
```

This creates **SettingsB** where the vault PDA is an authorized signer.

3. Attacker waits for the proposal's timelock to expire, then calls **execute_proposal** with:

```rust
remaining_accounts[0] = SettingsB  (attacker's settings)
   remaining_accounts[1] = PDA derived from SettingsB + next_tx_index
   remaining_accounts[2] = proposal PDA derived from SettingsB + next_tx_index
   remaining_accounts[3] = policy PDA derived from SettingsB + next_policy_seed
   remaining_accounts[4] = SQUADS_PROGRAM_ID
```

4. **split_policy_accounts** extracts these without checking against **vault.squads_settings**:

```rust
// action_execution.rs:195
   let squads_settings = &remaining_accounts[0];  // SettingsB, not validated
   // only squads_program is checked:
   require_keys_eq!(squads_program.key(), crate::SQUADS_PROGRAM_ID, ...);
```

5. **apply_add_policy** proceeds:

```rust
// manage_policy.rs:335-337
   let settings = load_squads_settings(squads_settings)?;  // loads SettingsB
   let policy_seed = next_policy_seed(&settings)?;          // seed from SettingsB
   validate_new_policy_account(squads_policy, squads_settings.key(), policy_seed)?;
   // validates against SettingsB, not SettingsA
```

6. Squads CPI succeeds -- vault PDA is a valid signer on SettingsB.

7. Policy is created on SettingsB. Proposal marked **Executed**. Vault's SettingsA is unchanged -- no policy was added.

Compare with the direct **AddPolicy** path which prevents this:

```rust
// manage_policy.rs:20-23
#[account(
    has_one = squads_settings @ ErrorCodeVaults::InvalidSquadsSettings,
)]
pub vault: Account<'info, ExponentStrategyVault>,
```

RECOMMENDATION

Add **vault.squads_settings** validation in **split_policy_accounts**:

```rust
fn split_policy_accounts<'info>(
    vault: &ExponentStrategyVault,
    remaining_accounts: &'info [AccountInfo<'info>],
) -> Result<(PolicyExecutionAccounts<'info>, &'info [AccountInfo<'info>])> {
    // ...
    let squads_settings = &remaining_accounts[0];
    require_keys_eq!(
        squads_settings.key(),
        vault.squads_settings,
        ErrorCodeVaults::InvalidSquadsSettings
    );
    // ...
}
```

FIX REVIEW NOTES

> **Developer Response:** **squads_settings** is no longer passed via **remaining_accounts**. It is now a dedicated parameter on **execute_proposal_actions**, validated upstream via **has_one** in the **ExecuteProposal** accounts struct.

> **Commit:** **8c9d5e8831a56017ab734be7c6d5b914e288199b**
> **Test:** https://github.com/exponent-finance/exponent-monorepo/pull/3645

> **Adevar Review:** Verified. The **has_one** constraint pins the Squads settings to the vault's stored value, eliminating the misdirection vector.

M10: Governance timelock computed from proposal creation instead of voting end will allow immediate execution after approval

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/state/proposal.rs:L97

RELEVANT SNIPPET

>_ proposal.rs — RUST

```rust
let executable_at = vault
    .reserves_config
    .proposal_executable_at(timestamp, timelock_seconds)
    .unwrap();
Self {
```

DESCRIPTION

The **ActionProposal::new()** function computes **executable_at** by passing the proposal's **creation timestamp** to **VaultConfig::proposal_executable_at()**. However, that function's parameter is named **finalized_at** and the vault's documentation states: "Timelock duration after voting period ends (seconds) / Policy becomes executable after voting_ends_at + timelock."

The actual calculation (for **Interval** mode):

RUST

```
executable_at = creation_time + max(timelock_seconds, lock_time_seconds)
```

The intended calculation (per documentation and parameter naming):

RUST

```
executable_at = voting_ends_at + max(timelock_seconds, lock_time_seconds)
```

With default configuration (48h voting period, 24h timelock):
- **executable_at = creation_time + 24h**
- **voting_ends_at = creation_time + 48h**

At finalization time (T+48h), **executable_at** (T+24h) is already 24 hours in the past. The proposal is immediately executable after finalization with zero effective timelock.

The purpose of a governance timelock is to give LP holders time to react (e.g., withdraw funds) between approval and execution. This bug eliminates that window for any configuration where **timelock_seconds < voting_period_seconds**, which includes the default.

RECOMMENDATION

Pass **voting_ends_at** instead of **timestamp** to **proposal_executable_at**:

RUST

```rust
// proposal.rs:95-98 — fix: use voting_ends_at
let executable_at = vault
    .reserves_config
    .proposal_executable_at(voting_ends_at, timelock_seconds)?;
```

This makes the timelock start after voting ends, matching the documented intent and the parameter name **finalized_at**.

FIX REVIEW NOTES

> **Developer Response:** **executable_at** is now correctly computed from **voting_ends_at** instead of the proposal creation timestamp.

> **Adevar Review:** Verified. The timelock now starts after the voting period ends, matching the documented intent.

M11: OptOut voters' withdrawals are unfillable due to zero-capacity account allocation in unstake_vote

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/proposals/unstake_vote.rs:L129

RELEVANT SNIPPET

>_ unstake_vote.rs — RUST

```
127:  if self.withdrawal_account.owner == &system_program::ID {
128:      let rent = Rent::get()?;
129:      let space = WithdrawalAccount::size_of(0);
130:      let required_lamports = rent.minimum_balance(space);
131:      let lamports_shortfall =
```

DESCRIPTION

When an LP holder votes OptOut on a proposal that gets approved, the **unstake_vote** instruction creates a **WithdrawalAccount** so the voter can exit before the proposal takes effect. However, the account is allocated with **WithdrawalAccount::size_of(0)** (unstake_vote.rs:130), which provides space for zero **WithdrawalTokenFill** entries.

RUST

```
if self.withdrawal_account.owner == &system_program::ID {
      let rent = Rent::get()?;
      let space = WithdrawalAccount::size_of(0);
      let required_lamports = rent.minimum_balance(space);
      let lamports_shortfall =
      required_lamports.saturating_sub(self.withdrawal_account.lamports());
      // ...
```

Compare to the normal **queue_withdrawal** path, which allocates **size_of(vault.token_entries.len())** - room for one fill per token entry.

When the manager calls **fill_withdrawal** on this opt-out withdrawal account, **record_token_fill** pushes a **WithdrawalTokenFill** (72 bytes: 32 mint + 32 source_token_account + 8 token_amount) onto the **fills** Vec. At the end of the instruction, Anchor serializes the account back to storage. The serialized data now exceeds the allocated space (**BASE_SIZE + 72 > BASE_SIZE**), and the transaction reverts.

This means opt-out withdrawal accounts can never be filled. Since they cannot be filled:
- **execute_withdrawal** requires **lp_filled > 0** (execute_withdrawal.rs:212), which is zero when no fills exist. Execution fails.

- **cancel_withdrawal** works (fills is empty) and returns LP to the voter, but they must then go through the normal **queue_withdrawal** flow which is subject to the standard lock period.

The opt-out mechanism exists as a convenience that bundles "unstake vote + queue withdrawal" into one step when an LP holder disagrees with an approved proposal. The resulting withdrawal is still subject to the same lock period and fill process as a normal withdrawal. With this bug, the mechanism is non-functional, voters must manually cancel and call **queue_withdrawal** separately, which produces the same outcome but requires extra transactions.

PROOF OF CONCEPT

1. Manager proposes a harmful action (e.g., increasing management fee).
2. Alice votes OptOut with 50,000 LP during the voting period.
3. Voting ends. Proposal is finalized as Approved.
4. Alice calls **unstake_vote**:
- Handler enters the opt-out withdrawal path (OptOut vote + Approved proposal)
- **init_opt_out_withdrawal_account** runs:

```rust
// unstake_vote.rs:130
     let space = WithdrawalAccount::size_of(0);  // zero fills capacity
```

- A **WithdrawalAccount** is created with **BASE_SIZE** bytes (164 bytes). No room for any fills.
- Alice's 50,000 LP remain locked in escrow.

5. Manager calls **fill_withdrawal** for Alice's opt-out withdrawal:
- **record_token_fill** pushes a fill onto the **fills** Vec:

```rust
// fill_withdrawal.rs:326
     withdrawal_account.fills.push(WithdrawalTokenFill {
         mint,
         source_token_account,
         token_amount,
     });
```

- Anchor tries to serialize the modified account back: needs **BASE_SIZE + 72** bytes, but only **BASE_SIZE** is allocated.
- Transaction reverts.

6. Alice cannot execute her withdrawal (no fills, **lp_filled == 0**).
7. Alice's only option is **cancel_withdrawal**, which returns her LP, then manually call **queue_withdrawal** to re-queue. Same outcome, extra transactions.

Normal **queue_withdrawal** for comparison:

```rust
// queue_withdrawal.rs:19
space = WithdrawalAccount::size_of(vault.token_entries.len()),
// Allocates room for fills -- fill_withdrawal works correctly
```

RECOMMENDATION

Change **unstake_vote.rs:130** to allocate the same fills capacity as the normal withdrawal path:

```rust
let space = WithdrawalAccount::size_of(vault.token_entries.len());
```

This requires passing the vault's **token_entries.len()** into **init_opt_out_withdrawal_account**. The vault is already available in the **UnstakeVote** accounts struct.

FIX REVIEW NOTES

> **Developer Response:** **WithdrawalAccount** space is now correctly allocated with capacity for fills.

> **Commit:** **9cfcf26c746b58df67b36ff7e3442510a88269ea**

> **Adevar Review:** Verified. Space allocation now matches the normal **queue_withdrawal** path.

M12: Zero lock time makes sentinel emergency takeover permanently unreachable, leaving stuck withdrawals with no recovery path

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/state/vault.rs:L203-L220

RELEVANT SNIPPET

>_ vault.rs RUST

```rust
}

pub fn can_make_sentinel_manager(&self, created_at: u32, current_time: u32) -> Result<bool> {
    let lock_time_seconds = match self.withdrawal_period {
        WithdrawalPeriodSettings::Interval { lock_time_seconds } => lock_time_seconds,
        // Sentinel takeover always uses interval-style timing, even for weekly withdrawals.
        WithdrawalPeriodSettings::WeeklySchedule { .. } => Self::SECONDS_PER_DAY,
    };

    let window_start = created_at
        .checked_add(lock_time_seconds)
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    let window_len = lock_time_seconds
        .checked_mul(SENTINEL_MANAGER_WINDOW_MULTIPLIER)
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    let window_end = window_start
        .checked_add(window_len)
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    Ok(current_time >= window_start && current_time <= window_end)
}

pub fn validate(&self) -> Result<()> {
```

DESCRIPTION

The sentinel escalation mechanism (**make_sentienel_manager**) allows a sentinel to replace all managers when a queued withdrawal remains unfilled past its lock period. The escalation window is derived from **lock_time_seconds**:

```
window_start = created_at + lock_time_seconds
window_end   = window_start + (lock_time_seconds * SENTINEL_MANAGER_WINDOW_MULTIPLIER)
```

The sentinel window duration is **lock_time_seconds * 3**. When **lock_time_seconds = 0** (the default **WithdrawalPeriodSettings::Interval** and a valid configuration for instant withdrawals), the window duration is **0 * 3 = 0**. The check becomes **current_time >= created_at && current_time <= created_at**, which requires **current_time == created_at**. This is impossible because **created_at** is set when the withdrawal is queued (a prior transaction), so by the time the sentinel calls **make_sentienel_manager**, **current_time > created_at** always holds.

This means the sentinel escalation mechanism is non-functional for every vault with a zero lock period, which is the default initialization path. The sentinel, the protocol's safety net against non-responsive managers, can never activate.

The root cause is that **can_make_sentinel_manager** derives the escalation window entirely from the withdrawal lock period. These are conceptually independent: the lock period governs how soon users can execute withdrawals, while the escalation window governs when the sentinel can intervene. Coupling them means a valid lock period configuration (zero, for instant withdrawals) silently disables an unrelated safety mechanism.

Additionally, at the opposite extreme, a malicious manager can set **lock_time_seconds** to a value above **u32::MAX / 3** (~1.4 billion) via **manage_vault_settings**, causing **checked_mul(SENTINEL_MANAGER_WINDOW_MULTIPLIER=3)** to overflow and revert the sentinel's transaction with **MathOverflow**. Combined with extreme **lock_time_seconds** making **proposal_executable_at** return effectively infinite delays, this also locks out governance, preventing LP holders from reverting the change.

PROOF OF CONCEPT

Default case (affects all default-configured vaults):

1. Vault is initialized with default settings. **lock_time_seconds = 0** (instant withdrawals, no lock).
2. Vault accumulates $1M AUM. A sentinel is configured for emergency protection.
3. Manager becomes non-responsive.
4. User queues a withdrawal at time T. Manager never fills it.
5. Sentinel calls **make_sentienel_manager** at time T+1:

```rust
// can_make_sentinel_manager(created_at=T, current_time=T+1):
let lock_time_seconds = 0;
let window_start = T + 0;       // = T
let window_len = 0 * 3;         // = 0
let window_end = T + 0;         // = T
// check: T+1 >= T && T+1 <= T  -> false
```

6. Sentinel cannot escalate. The withdrawal remains stuck.

Malicious manager case:

1. Manager calls **manage_vault_settings** with **SetWithdrawalPeriod(Interval { lock_time_seconds: 1_500_000_000 })** (~47 years, within u32 range).
2. Sentinel calls **make_sentienel_manager**:

```rust
let window_len = 1_500_000_000.checked_mul(3); // overflow -> MathOverflow error
```

3. Transaction reverts. Sentinel is permanently bricked.

RECOMMENDATION

Two changes are needed:

1. Cap **lock_time_seconds** in **validate()** to a reasonable maximum (e.g., 30 days). This prevents a malicious manager from setting an extreme value that overflows arithmetic or makes withdrawals and governance unreachable. The Interval variant currently has no validation at all. With the cap, **created_at + lock_time_seconds + SENTINEL_GRACE_PERIOD** cannot overflow.

2. Replace the expiring window with a one-sided threshold:

```
sentinel_allowed = current_time >= created_at + lock_time_seconds + SENTINEL_GRACE_PERIOD
```

Where **SENTINEL_GRACE_PERIOD** is a protocol constant (e.g., 24 hours). This preserves the intent: the manager gets the full lock period plus additional grace time to fill the withdrawal before the sentinel can step in. But once that deadline passes, the sentinel can escalate at any time, there is no reason this ability should expire.

FIX REVIEW NOTES

> **Developer Response:** **can_make_sentinel_manager** now uses a one-sided grace period instead of a bounded window, ensuring the sentinel can always escalate after the grace period expires. Additionally, the fix includes logic to ensure the manager has filled all withdrawals in the last window before continuing **interaction_validations**.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3644

> **Adevar Review:** The one-sided grace period correctly resolves the original sentinel unreachability issue. However, the newly added constraint that blocks manager interactions when pending withdrawals exist raises a concern: the manager sometimes needs to withdraw from strategies in order to fill queued withdrawals, creating a circular dependency.

M13: Stale validation view in post-hook batch validation will bypass reserves constraint, allowing manager to deplete all liquid reserves

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/admin/validate_interaction.rs:L174-L184

RELEVANT SNIPPET

>_ validate_interaction.rs — RUST

```
172:     let accounts = ctx.remaining_accounts;
173:
174:     for ix in &inner_instructions.instructions {
175:         let mutations = hook_validation::validate_instruction(
176:             &validation_view,
177:             ix.program_id_index,
178:             &ix.account_indexes,
179:             &ix.data,
180:             accounts,
181:             &tracked,
182:         )?;
183:         hook_validation::apply_vault_mutations(vault, &mut validation_view, &mutations
    )?;
184:     }
185: }
186:
```

DESCRIPTION

When a vault manager executes operations through Squads, the Squads program first executes the inner instructions (e.g., depositing into Kamino reserves, minting SY tokens), then CPIs into the exponent vault's **validate_interaction_hook** (discriminator **[21]**) as a post-hook. If the hook reverts, the entire transaction reverts, making it an effective gate on what the manager can do.

The hook validates each inner instruction against the vault's reserves policy. For each instruction in the batch, the dispatch is:

1. **validate_interaction.rs:174-182** -- loop over inner instructions, calling **hook_validation::validate_instruction(&validation_view, ...)**
2. **hook_validation.rs:87-88** -- delegates to **integrations::validate_instruction(vault, ...)**
3. **utils.rs:13-73** -- dispatches to the per-program validator (e.g., **kamino::validate**, **token::validate**, **standard_program::validate**)
4. Each validator calls **vault.verify_aum_change(aum_diff)** on the shared **VaultValidationView**

5. **validation_view.rs:18-30** -- checks **self.aum_in_positions_in_base + aum_diff <= self.effective_free_aum_in_base**

The **VaultValidationView** is built once at **hook_validation.rs:58-77** before the loop starts.

```rust
let mut validation_view = hook_validation::build_vault_validation_view(
    vault,
    &exponent_prices,
    current_slot,
    aum_locked,
  )?;
```

The **aum_in_positions_in_base** field is read from **vault.financials.aum_in_base_in_positions**, which is the vault's stored snapshot. This value is stale because the inner instructions already executed are interactions with external programs (Kamino, Pendle-style SY, etc.) that move tokens but never update the exponent vault's stored AUM fields.

When the loop processes multiple inner instructions, each calls **verify_aum_change** on the same **VaultValidationView**. The view's **aum_in_positions_in_base** is never updated between iterations. **apply_vault_mutations** (line 183) only modifies orderbook offer tracking -- the **VaultMutation** enum (**validation_view.rs:84-87**) has no variant for AUM changes:

```rust
pub enum VaultMutation {
    TrackOrderbookOffer { orderbook: Pubkey, offer_index: u32 },
    UntrackOrderbookOffer { orderbook: Pubkey, offer_index: u32 },
}
```

So every instruction independently checks its deployment against the same stale, pre-batch position total. If the remaining deployment capacity is **C**, each of N instructions can individually deploy up to **C**, for a total of **N * C**.

This only manifests when the Squads sync transaction contains more than one inner instruction that deploys AUM into positions. A single-instruction transaction is unaffected.

PROOF OF CONCEPT

Setup: Vault with $1M AUM, **reserves_share_bp = 2000** (20% reserves), no current positions. The vault has a Kamino reserve and a Pendle SY mint configured as strategy positions.
- **effective_free_aum = $800K**
- **aum_in_positions_in_base = $0**
- Remaining deployment capacity: **C = $800K**

1. Manager creates a Squads sync transaction with 2 inner instructions:
- Instruction 1: Kamino **deposit_reserve_liquidity** -- deposit $800K into a Kamino lending reserve
- Instruction 2: Standard program **mint_sy** -- mint $200K worth of SY tokens via a Pendle-style yield position

2. Squads executes both instructions. Vault now has $0 liquid, $1M deployed across Kamino and SY positions.

3. Post-hook fires. **VaultValidationView** is built with **aum_in_positions_in_base = $0** (stale -- neither the Kamino program nor the SY program update the vault account's stored financials).

4. Loop iteration 1 -- **kamino::validate_deposit_liquidity** for the Kamino deposit:

```
verify_aum_change($800K):
     current_in_positions = $0  (stale)
     new_in_positions = $0 + $800K = $800K
     $800K <= $800K -> passes
```

5. Loop iteration 2 -- **standard_program::validate_mint_sy** for the SY mint:

```
verify_aum_change($200K):
     current_in_positions = $0  (still stale, never updated)
     new_in_positions = $0 + $200K = $200K
     $200K <= $800K -> passes
```

6. Hook succeeds. Total deployed: $1M. Reserves: $0. The 20% reserves constraint is fully bypassed.

With correct cumulative tracking, iteration 2 should check:

```
current_in_positions = $800K  (accumulated from iteration 1)
new_in_positions = $800K + $200K = $1M
$1M > $800K -> ReservesExceeded
```

RECOMMENDATION

After each instruction validation in the loop, accumulate the validated **aum_diff** into the **VaultValidationView**'s **aum_in_positions_in_base** field. This can be done by adding a new **VaultMutation** variant (e.g., **AumPositionChange { aum_diff: i64 }**) and applying it in **apply_vault_mutations**, or by having each validator return the **aum_diff** and updating the view directly in the loop. This ensures each subsequent instruction sees the cumulative effect of all prior instructions in the batch.

FIX REVIEW NOTES

> **Developer Response:** AUM diff is now accumulated into **cumulative_aum_diff** across loop iterations.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3643

> **Adevar Review:** Verified. Each instruction validation now sees the cumulative effect of all prior instructions in the batch.

L01: Cancellation in project_staged_sy loses precision on projected yield calculation

LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/position_values/orderbook.rs:L168-L169

RELEVANT SNIPPET

>_ orderbook.rs — RUST

```
166:     .to_f64()
167:     .ok_or(ErrorCodeVaults::MathOverflow)?;
168: let rate_delta = 1.0 / last_seen_sy_index - 1.0 / current_sy_rate_f64;
169: let interest_earned = (rate_delta * user_escrow.staked_yt_amount as f64).floor() as u6
       4;
170:
171: user_escrow
```

DESCRIPTION

project_staged_sy computes accrued YT yield by subtracting two reciprocals of nearly-equal rates:

RUST

```
let rate_delta = 1.0 / last_seen_sy_index - 1.0 / current_sy_rate_f64;
let interest_earned = (rate_delta * user_escrow.staked_yt_amount as f64).floor() as u64;
```

Each 1.0 / rate division truncates the true value to f64's 15 significant digits.

When last_seen_sy_index and current_sy_rate are close (normal during slow yield accrual), the subtraction cancels most shared digits, leaving rate_delta with far fewer significant digits. Multiplying this imprecise rate_delta by a large staked_yt_amount amplifies the truncated digits into a token-level difference in interest_earned.

Coverage-guided fuzzing compared the current formula against the algebraically equivalent restructured form staked_yt × (new_rate - old_rate) / (old_rate × new_rate), which avoids cancellation by subtracting the original rate values directly instead of two intermediate division results.

Results at 12 decimals (up to 10M tokens staked, 694 tests): 1,280 violations. Worst case: **staked_yt = 8.7M tokens**, rates differ by **0.000000065**, result off by **0.000000001408** tokens.

Results at 9 decimals (up to 16.9B tokens staked, 837 tests): 1,520 violations. Worst case: **staked_yt = 16.9B tokens**, rates differ by **0.0001%**, result off by **0.000003149** tokens.

The per-calculation error is dust-level but accumulates across repeated AUM recalculations for every vault holding YT positions with active yield accrual.

RECOMMENDATION

Restructure the formula to avoid subtracting two truncated reciprocals:

```rust
let rate_delta = (current_sy_rate_f64 - last_seen_sy_index)
    / (last_seen_sy_index * current_sy_rate_f64);
```

Same math, one division at the end instead of two divisions before the subtraction. Preserves full f64 precision regardless of how close the rates are.

DEVELOPER RESPONSE

Acknowledged by the Exponent team.

L02: Double flooring in AUM and LP calculation causes exchange rate to decrease on deposits

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/state/vault.rs:L375-L381

RELEVANT SNIPPET

>_ vault.rs — RUST

```
373:  );
374:
375:  let numerator = (lp_supply as u128)
376:      .checked_mul(amount_base_in as u128)
377:      .ok_or(ErrorCodeVaults::MathOverflow)?;
378:
379:  let lp_out = numerator
380:      .checked_div(self.total_aum() as u128)
381:      .ok_or(ErrorCodeVaults::MathOverflow)? as u64;
382:
383:  require!(lp_out > 0, ErrorCodeVaults::AmountTooSmall);
```

DESCRIPTION

The exchange rate (**totalAssets / totalSupply**) decreases after certain deposit sequences, violating the invariant that deposits should never dilute existing LP holders.

Two consecutive floor operations interact to favor the depositor. First, **recalculate_aum_in_base** floors the result of **token_balance × price** when storing **aum_in_base**:

RUST

```
// aum.rs line 496
self.financials.aum_in_base = aum_in_base.floor_u64();
```

This produces a **total_aum** that is slightly smaller than the true value. Then **lp_out_from_base** uses this floored value as the denominator:

RUST

```
// vault.rs — lp_out_from_base
lp_out = base_amount_in * lp_supply / total_aum
```

Dividing by a smaller denominator inflates **lp_out** by up to 1 LP unit. The depositor receives slightly more shares than their fair proportion, reducing the per-share value for all existing holders.

Coverage-guided fuzzing confirmed this across 213,461 deposits at price levels from $3 to $10,000, producing 20,883 invariant violations. The rate decrease is ~0.0000000001 per deposit, detectable only via integer cross-multiplication (**assets_now × supply_prev < assets_prev × supply_now**).

RECOMMENDATION

Floor **lp_out** after the full calculation instead of flooring **total_aum** before it.

Replace the intermediate **floor_u64()** in **recalculate_aum_in_base** with the precise **Number** value when passing to **lp_out_from_base**, so only one floor occurs at the final LP output step.

FIX REVIEW NOTES

> **Developer Response:** The AUM calculation now returns a **Number** type instead of flooring to **u64**, preserving full precision for the LP output calculation.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3651

> **Adevar Review:** Verified. The non-floored **Number** value is now used in the LP calculation, eliminating the double-floor interaction.

L03: Mandatory offer_idx in post_offer hook validation causes transaction revert when counter offers fully fill the order

LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/interaction_validations/orderbook.rs:L74-L75

RELEVANT SNIPPET

>_ orderbook.rs — RUST

```
72:
73: let orderbook_key = *accounts[orderbook_account_index].key;
74: let offer_idx =
75:     deserialize_post_offer_offer_idx(data)?.ok_or(ErrorCodeVaults::UnsupportedInstruct
      ion)?;
76:
77: let entry = find_orderbook_entry(vault, orderbook_key)?;
```

DESCRIPTION

The interaction validation for **post_offer** and **wrapper_post_offer** requires the caller to specify **offer_idx: Some(X)**:

RUST

```
let offer_idx = args.offer_idx.ok_or(ErrorCodeVaults::UnsupportedInstruction)?;
```

When **post_offer** fills all counter offers and no residual remains, it returns **None** for the offer index:

RUST

```
// post_offer in exponent_orderbook
let offer_idx = if remaining_amount > self.configuration_options.threshold_amount {
    let new_offer_idx = self.add_offer(...).unwrap();
    remaining_amount = 0;
    Some(new_offer_idx)
} else {
    None  // fully filled, no residual offer created
};
```

The orderbook then checks the caller's prediction against this result:

```rust
if let Some(expected_idx) = offer_idx {
    require!(
        post_offer_output.offer_idx == Some(expected_idx),
        OrderbookValidationsErrors::OfferIdxMismatch
    );
}
```

Since the caller specified **Some(X)** but the output is **None**, the check becomes **None == Some(X)**, which fails with **OfferIdxMismatch**. The inner instruction reverts and the post-hook never runs.

This means any **post_offer** that gets fully filled as a taker (a beneficial outcome where all counter offers matched) is rejected. The manager cannot atomically fill counter offers and post the remainder as a limit order when there's a chance the full amount matches.

A related issue: if the orderbook state changes between when the manager reads it and when the transaction lands (e.g., another user posts or removes an offer), the predicted index may not match the actual allocation. This causes the same **OfferIdxMismatch** revert.

PROOF OF CONCEPT

1. Orderbook has 1000 SY worth of counter offers at matching prices.
2. Manager builds Squads tx: **wrapper_post_offer(amount_base=800, offer_idx: Some(5))**.
3. All 800 is filled against counter offers. No residual offer. **post_offer_output.offer_idx = None**.
4. Orderbook check: **None != Some(5)**. Reverts with **OfferIdxMismatch**.
5. Entire Squads tx rolls back. All counter-offer fills are undone.

RECOMMENDATION

Since the hook runs post-execution (the on-chain orderbook state is already mutated), the hook could read the post-execution orderbook account to identify whether a new offer was actually created, rather than relying on the caller's prediction. This would handle both full fills (no offer -> skip tracking) and partial fills (verify and track the actual index). However, this introduces additional complexity in parsing the orderbook's offer tree.

Alternatively, document this as a known limitation and restrict vault managers to using **market_offer** for taker fills and **post_offer** only for limit orders where full fills are not expected.

DEVELOPER RESPONSE

> **Developer Response:** Acknowledged. The Exponent team classified this as Informational severity and recommends using market orders for taker fills, as a proper fix would require reading from post-execution orderbook state, introducing additional complexity.

> **Adevar Review:** The use of market orders is a reasonable operational workaround.

L04: Kamino Withdraw and Repay Instructions Missing From Hook Validator

LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/interaction_validations/kamino.rs:L58-L69

RELEVANT SNIPPET

>_ kamino.rs — RUST

```rust
    };

    match kind {
        KaminoInstructionKind::RefreshReserve => Ok(Some(Vec::new())),
        KaminoInstructionKind::RefreshObligation => {
            validate_refresh_obligation(vault, account_indexes, accounts, tracked)
        }
        KaminoInstructionKind::DepositReserveLiquidity => {
            validate_deposit_liquidity(vault, data, account_indexes, accounts, tracked
)
        }
        KaminoInstructionKind::BorrowObligationLiquidity => {
            validate_borrow_liquidity(vault, account_indexes, accounts, tracked)
        }
    }
}

```

DESCRIPTION

The Kamino hook validator only recognizes **refresh_reserve**, **refresh_obligation**, **deposit_reserve_liquidity**, and **borrow_obligation_liquidity**.

Withdraw (**withdraw_obligation_collateral_and_redeem_reserve_collateral**) and repay (**repay_obligation_liquidity**) hit the **_ => return Ok(None)** fallthrough, and since no other validator handles the Kamino program ID, both are rejected as **UnsupportedInstruction**.

This makes Kamino positions one-directional through the hooked flow: the manager can deposit and borrow but cannot withdraw collateral or repay debt. To unwind a position or improve health factor before liquidation, the manager must use a policy without the post-hook, bypassing all hook validation.

Kamino itself enforces **token::authority = owner** on destination accounts for both operations, so the manager cannot redirect funds to an external address. The direct security risk is limited, but the operational gap forces reliance on unvalidated policies for routine position management.

RECOMMENDATION

Add withdraw and repay discriminators to the Kamino validator.

FIX REVIEW NOTES

> **Developer Response:** The hook validator now supports both withdraw and repay instructions.

> **Commit:** 4d4551b648db0f9bb894df9997010bd5a6bdc045

> **Adevar Review:** Verified. Both instruction discriminators are now recognized by the Kamino validator.

5. Enhancement Opportunities

E01: Validate offer_idx is non-zero in add_offer_idx to prevent NodeAllocator underflow panic

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/admin/manager_update_position.rs:L266-L279

RELEVANT SNIPPET

>_ manager_update_position.rs — RUST

```rust
}

fn add_offer_idx(
    vault: &mut ExponentStrategyVault,
    orderbook: Pubkey,
    offer_idx: u32,
) -> Result<()> {
    let entry = find_orderbook_entry_mut(vault, orderbook)?;

    require!(
        !entry.offer_idx_vec.contains(&offer_idx),
        ErrorCodeVaults::OfferIdxAlreadyExists
    );

    entry.offer_idx_vec.push(offer_idx);
    Ok(())
}
```

DESCRIPTION

add_offer_idx accepts index 0 (the sokoban NodeAllocator SENTINEL value). If pushed into offer_idx_vec, the next recalculate_aum_in_base calls raw_orderbook.offers.get(0) which underflows to nodes[(0-1) as usize], an out-of-bounds panic that bricks all vault operations (deposits, withdrawals, fills). Recovery requires governance to remove the entire orderbook entry.

The hook path is protected by the orderbook program's own offer_idx mismatch check, but add_offer_idx via manager_update_position has no such external guard.

POTENTIAL BENEFIT

Prevents an accidental self-DoS where a privileged role bricks the vault by passing an invalid offer index. Defense-in-depth against sokoban NodeAllocator's 1-based indexing where 0 is a reserved sentinel value.

RECOMMENDATION

Add require!(offer_idx != 0, ErrorCodeVaults::InvalidOfferIdx) in add_offer_idx.

FIX REVIEW NOTES

> **Developer Response:** `offer_idx=0` is now disallowed in `add_offer_idx`.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3653

> **Adevar Review:** Verified.

E02: Redundant rejection threshold recalculation in is_proposal_rejected

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/proposals/finalize_proposal.rs:L58-L63

RELEVANT SNIPPET

>_ finalize_proposal.rs RUST

```
56:     .ok_or(ErrorCodeVaults::MathOverflow)?;
57:
58: let result = if ctx
59:     .accounts
60:     .vault
61:     .proposal_vote_config
62:     .is_proposal_rejected(effective_reject_votes, total_lp_supply)?
63: {
64:     ProposalStatus::Rejected
65: } else {
```

DESCRIPTION

finalize_proposal calculates rejection_required via calculate_rejection_required(total_lp_supply), then immediately calls is_proposal_rejected(effective_reject_votes, total_lp_supply), which internally recalculates the same threshold from the same total_lp_supply.

The threshold is computed twice with identical inputs in the same instruction.

RUST

```
let rejection_required = ctx.accounts.vault.proposal_vote_config
    .calculate_rejection_required(total_lp_supply)?;

// ... then:

let result = if ctx.accounts.vault.proposal_vote_config
    .is_proposal_rejected(effective_reject_votes, total_lp_supply)? // recalculates threshold
```

The first rejection_required value is only used in the emitted event, not passed to is_proposal_rejected.

POTENTIAL BENEFIT

Eliminates one redundant calculate_rejection_required computation per finalization. Simplifies the code path by making the data flow explicit: compute threshold once, use it for both the rejection check and the event.

RECOMMENDATION

Pass the precomputed **rejection_required** to the rejection check instead of recomputing it:

```rust
let rejected = effective_reject_votes >= rejection_required;
let result = if rejected { ProposalStatus::Rejected } else { ProposalStatus::Approved };
```

DEVELOPER RESPONSE

Acknowledged by the Exponent team.

E03: Division before multiplication in get_sy_in_offer loses f64 precision on large offer amounts

LOCATION

exponent-monorepo/e8fdc0f8/solana/libraries/exponent_vaults_integrations/src/position_values/orderbook.rs:L181-L183

RELEVANT SNIPPET

>_ orderbook.rs RUST

```
179:
180: if offer.is_virtual() {
181:     if offer_type == OfferType::BuyYt {
182:         let pt_to_sy_rate = pt_price_in_quote_asset / sy_exchange_rate;
183:         ((offer.amount as f64) * pt_to_sy_rate).floor() as u64
184:     } else {
185:         offer.amount
```

DESCRIPTION

get_sy_in_offer divides first to produce a ratio, then multiplies by the offer amount:

RUST

```
let pt_to_sy_rate = pt_price_in_quote_asset / sy_exchange_rate;
((offer.amount as f64) * pt_to_sy_rate).floor() as u64
```

The intermediate ratio loses fractional bits in f64's 53-bit mantissa. Multiplying by a large offer.amount (above 2^53) amplifies the truncation. Fuzzing confirmed divergence of up to 1,024 raw SY units between divide-first and multiply-first orderings across 14,318 test cases. Dollar impact is ~$0.001 per offer max.

This is the only function in the codebase that uses f64 for AUM-critical arithmetic. All other value conversions use Number (U256 with 1e12 precision).

POTENTIAL BENEFIT

Eliminates intermediate ratio truncation. Aligns with the precision standard.

RECOMMENDATION

Multiply before dividing:

RUST

```
((offer.amount as f64) * pt_price_in_quote_asset / sy_exchange_rate).floor() as u64
```

FIX REVIEW NOTES

> **Developer Response:** Multiplication is now performed before division.

> **PR:** https://github.com/exponent-finance/exponent-monorepo/pull/3650

> **Adevar Review:** Verified.

E04: Remove Redundant updated_at Write in execute_withdrawal

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/execute_withdrawal.rs:L237

RELEVANT SNIPPET

>_ execute_withdrawal.rs RUST

```
235:  vault.recalculate_aum_in_base(&exponent_prices, ctx.remaining_accounts)?;
236:
237:  ctx.accounts.withdrawal_account.updated_at = now();
238:
239:  let event = ExecuteWithdrawalEvent {
```

DESCRIPTION

execute_withdrawal writes **withdrawal_account.updated_at = now()**, immediately before the **WithdrawalAccount** is closed (via Anchor's **close = owner** constraint).

The account ceases to exist after the instruction completes, so the timestamp is never read.

The field is never read on-chain by any instruction.

POTENTIAL BENEFIT

Removing the write in **execute_withdrawal** eliminates a wasted account serialization step on a soon-to-be-closed account

RECOMMENDATION

Remove the **updated_at** assignment in **execute_withdrawal**:

DEVELOPER RESPONSE

Acknowledged by the Exponent team.

E05: Prepopulate Authorized Price Manager List Before Initialization

LOCATION

exponent-monorepo/e8fdc0f8/solana/programs/exponent_vaults/src/instructions/admin/initialize_prices.rs:L127-L130

RELEVANT SNIPPET

>_ initialize_prices.rs RUST

```
125: }
126:
127: let mut exponent_prices = load_exponent_prices_mut(&exponent_prices_account)?;
128: if should_initialize || exponent_prices.price_managers_len == 0 {
129:     exponent_prices.initialize(ctx.accounts.manager.key())?;
130: }
131:
132: Ok(())
```

DESCRIPTION

initialize_exponent_prices_if_ready sets the caller (**ctx.accounts.manager**) as the first price manager unconditionally when the **ExponentPrices** account reaches full size:

RUST

```
// initialize_prices.rs:171-173
if should_initialize || exponent_prices.price_managers_len == 0 {
    exponent_prices.initialize(ctx.accounts.manager.key())?;
}
```

The **InitializePrices** accounts struct has no role-based constraint on **manager**. It is simply a **Signer** who pays for account creation. This means whoever calls the final realloc transaction (the one that brings the account to full size) becomes the first price manager. There is no pre-configured allowlist of authorized price managers to validate against.

A more robust approach would be to define the authorized price manager set on the vault or a separate config account before initialization, and then require the caller of **initialize_exponent_prices_if_ready** to be present on that list.

POTENTIAL BENEFIT

Enforcing a pre-configured price manager allowlist at initialization time ensures that only explicitly authorized addresses can become price managers.

This eliminates the implicit trust assumption that the correct party will happen to call the final realloc, and prevents accidental or adversarial assignment of the price manager role during deployment.

RECOMMENDATION

Store the intended price manager address(es) in a config account or vault state before **ExponentPrices** initialization. Then validate the caller against them

DEVELOPER RESPONSE

Acknowledged by the Exponent team.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: Fuzzing Harness Source Code

The fuzzing harness below was developed using Crucible, internal coverage-guided fuzzing framework for Solana programs. Crucible is not yet open source, and we are unable to share the binary at this time. The harness source code is included here for reference and reproducibility.

Instruction for fuzzing set-up

1. Install Anchor Fuzz via **cargo install --path crates/crucible-fuzz-cli**
2. Retrieve the code **.so** & **IDL** files and import it to the fuzzer repo.
3. Test via crucible **run PROJECT_NAME invariant_test --dry-run**

```rs
// Crucible fuzzer framework — provides TestContext, fuzz_fixture, invariant_test macros, fuzz_assert_*
use crucible_fuzzer::*;
use solana_keypair::Keypair;
use solana_pubkey::Pubkey;
use solana_signer::Signer;
use crucible_fuzzer::anchor_lang::system_program;
use crucible_fuzzer::anchor_spl::token::spl_token;
use std::{rc::Rc, collections::HashMap};
use solana_instruction::AccountMeta;

// Reads the Anchor IDL at compile time and generates:
//   exponent_vaults::instruction::* — instruction data structs (know how to serialize discriminator + args)
//   exponent_vaults::accounts::*   — account context structs (know account order, writable/signer flags)
//   exponent_vaults::types::*      — program's custom types (VaultType, VaultRoles, TokenEntry, etc.)
//   exponent_vaults::ID            — program pubkey
crucible_idl_gen::declare_fuzz_program!("idls/exponent_vaults.json");

use exponent_vaults::instruction;
use exponent_vaults::accounts;

const DEBUG: bool = true;

// ============================================================================
// Seeds & Constants
// ============================================================================

// PDA seeds — copied from the program source. Same seeds + program_id = same deterministic address.
const PRICES_SEED: &[u8] = b"exponent_prices"; // Global price oracle account PDA seed
const VAULT_SEED: &[u8] = b"vault";            // Per-vault PDA seed (combined with seed_id)

// Squads v4 multisig program — Exponent uses it to custody vault funds.
// The vault PDA becomes the sole signer on a Squads smart account.
```

... continued RS

```rust
const SQUADS_PROGRAM_ID_STR: &str = "SMRTzfY6DfH5ik3TKiyLFfXexV8uSG3d2UksSCYdunG";

// ExponentPrices account total size (discriminator + data). Fits in one realloc (<10,239 byte limit).
const EXPONENT_PRICES_SIZE: usize = 8096;

// First 8 bytes of the Squads ProgramConfig account — Anchor uses this to verify account type.
// Equivalent to SHA256("account:ProgramConfig")[..8].
const SQUADS_PROGRAM_CONFIG_DISCRIMINATOR: [u8; 8] = [196, 210, 90, 231, 144, 149, 140, 63];

// LiteSVM can panic internally (e.g., program cache desync after snapshot restore).
// These macros catch the panic and convert it to an error so the fuzzer keeps running.
macro_rules! safe_send {
    ($builder:expr) => {
        match std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| $builder.send())) {
            Ok(result) => result,
            Err(_) => Err(anyhow::anyhow!("litesvm panic during send")),
        }
    };
}

macro_rules! safe_send_batch {
    ($ctx:expr) => {
        match std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| $ctx.send_batch())) {
            Ok(result) => result,
            Err(_) => Err(anyhow::anyhow!("litesvm panic during send_batch")),
        }
    };
}

// ============================================================================
// Data Structures
// ============================================================================

// Per-user state. Rc<Keypair> because the fixture must implement Clone (snapshot/restore per iteration).
#[derive(Clone)]
struct UserData {
    keypair: Rc<Keypair>,
    token_accounts: HashMap<Pubkey, Pubkey>, // mint pubkey → user's token account for that mint
    lp_token_account: Pubkey,                // user's account for LP shares
}

// Main fixture — holds the entire test state. Cloned at start of each fuzzer iteration.
// All Pubkey fields are just addresses (the actual data lives in ctx.svm).
#[derive(Clone)]
struct ExponentFuzz {
    ctx: TestContext,            // LiteSVM wrapper — the fake Solana
    program_id: Pubkey,          // exponent_vaults program address
    squads_program_id: Pubkey,   // Squads v4 program address
    admin: Rc<Keypair>,          // Vault manager — can fill withdrawals, collect fees, etc.
```

... continued RS

```rs
    // Protocol accounts (addresses only — data is in ctx.svm)
    exponent_prices: Pubkey,      // Global oracle price storage PDA
    vault: Pubkey,                // ExponentStrategyVault PDA — the core vault state
    vault_bump: u8,               // PDA bump seed (needed for vault to sign CPIs)
    seed_id: [u8; 8],            // Unique identifier for this vault instance
    mint_lp: Pubkey,              // LP token mint — users receive these on deposit
    token_lp_escrow: Pubkey,      // Holds LP tokens locked during withdrawal queue +
  governance votes
    fee_treasury: Pubkey,         // Receives protocol fee LP on withdrawal fills
    squads_settings: Pubkey,      // Squads multisig settings account
    squads_vault: Pubkey,         // Squads sub-account 0 — actually holds custodied tokens
    squads_token_account: Pubkey, // Squads vault's SOL token account (deposits land here)
    vault_token_account: Pubkey,  // Vault PDA's SOL token account (intermediate for
  withdrawals)
    squads_policy_transfer: Pubkey, // Squads transfer policy PDA (signs withdrawal CPI)
    // Oracle
    mock_oracle: Pubkey,          // Mock Pyth oracle for SOL at $298
    pyth_program_id: Pubkey,      // Pyth Receiver program ID
    price_interface_accounts_pda: Pubkey, // PDA storing oracle account references
    // Token mints
    underlying_mint: Pubkey,      // Base denomination (USDC, 6 decimals) — AUM measured in
  this
    sol_mint: Pubkey,             // Deposit token (SOL-like, 9 decimals, $298) — users deposit
   this
    // Admin accounts (for tracking in invariants)
    admin_token_account: Pubkey, // Admin's SOL token account
    // Invariant tracking
    prev_rate_num: u128,          // previous exchange rate numerator (totalAssets)
    prev_rate_den: u128,          // previous exchange rate denominator (totalSupply)
    // Users
    users: Vec<UserData>,         // 3 test users with funded token + LP accounts
}

#[fuzz_fixture]
impl ExponentFuzz {
    pub fn setup() -> Self {
        let mut ctx = TestContext::new();
        let program_id = exponent_vaults::ID;
        let squads_program_id: Pubkey = SQUADS_PROGRAM_ID_STR.parse().unwrap();

        // Resolve .so paths relative to the binary's directory (needed for FuzzCorp
  containers)
        let bin_dir = std::env::current_exe()
            .ok()
            .and_then(|p| p.parent().map(|d| d.to_path_buf()))
            .unwrap_or_default();

        // ============================================================
        // STEP 1: Load program binaries into LiteSVM
        // ============================================================
        // Like deploying contracts. LiteSVM will execute these .so files when
        // transactions target their program IDs. Both are needed because
        // initialize_vault does CPI into Squads to create the multisig wallet.
```

... continued RS

```rs
        ctx.add_program(&program_id, bin_dir.join("exponent_vaults.so").to_str().unwrap())
            .expect("Failed to load exponent_vaults program");
        eprintln!("[SETUP] Loaded exponent_vaults: {}", program_id);

        ctx.add_program(&squads_program_id, bin_dir.join("squads_smart_account_program.so").to
_str().unwrap())
            .expect("Failed to load squads program");
        eprintln!("[SETUP] Loaded squads: {}", squads_program_id);

        // Token-2022 needed by execute_withdrawal_from_reserves (Account<Token2022>
constraint)
        let token_2022_id: Pubkey = "TokenzQdBNbLqP5VEhdkAS6EPFLC1PHnBqCXEpPxuEb".parse().unwr
ap();
        ctx.add_program(&token_2022_id, bin_dir.join("token_2022.so").to_str().unwrap())
            .expect("Failed to load Token-2022 program");
        eprintln!("[SETUP] Loaded Token-2022: {}", token_2022_id);

        // ================================================================
        // STEP 2: Create admin account with SOL
        // ================================================================
        // 500 SOL because initialize_vault creates ~10 accounts (vault PDA, LP mint,
        // LP escrow, Squads settings, Squads vault, 2 policies, 2 transactions, 2 proposals)
        // and each requires rent-exempt lamports.
        let admin = Rc::new(Keypair::new());
        ctx.create_account()
            .pubkey(admin.pubkey())
            .lamports(500_000_000_000)
            .owner(system_program::ID)
            .create()
            .unwrap();
        eprintln!("[SETUP] Admin: {}", admin.pubkey());

        // ================================================================
        // STEP 3: Initialize ExponentPrices
        // ================================================================
        // ExponentPrices is a GLOBAL SINGLETON — all vaults share one price oracle account.
        // It's a zero-copy account storing up to 32 price feeds with reference counting.
        // Must be initialized before any vault can be created (vaults validate price chains).
        let (exponent_prices, _prices_bump) = Pubkey::find_program_address(
            &[PRICES_SEED],
            &program_id,
        );
        eprintln!("[SETUP] ExponentPrices PDA: {}", exponent_prices);

        // This program instruction creates the account at the PDA address, stamps the
        // discriminator, and registers the caller as the first price manager.
        let result = ctx.program(program_id)
            .call(instruction::InitializePrices {})
            .accounts(accounts::InitializePrices {
                manager: admin.pubkey(),
                exponent_prices,
                system_program: system_program::ID,
            })
```

... continued RS

```
            .signers(&[&*admin])
            .send();

        match &result {
            Ok(outcome) if outcome.is_success() => {
                eprintln!("[SETUP] InitializePrices SUCCESS");
            }
            Ok(outcome) => {
                eprintln!("[SETUP] InitializePrices FAILED: {:?}", outcome);
                for log in outcome.logs() { eprintln!("  {}", log); }
                panic!("Setup failed: InitializePrices");
            }
            Err(e) => {
                eprintln!("[SETUP] InitializePrices SEND ERROR: {:?}", e);
                panic!("Setup failed: InitializePrices");
            }
        }

        // ================================================================
        // STEP 4: Create token mints
        // ================================================================
        // Two mints: USDC as base denomination (AUM measured in USDC),
        // SOL as the deposit token (what users actually deposit).
        // This creates interesting rounding in price conversions ($298 SOL → USDC).

        // Base denomination: USDC (6 decimals, $1). AUM is measured in this.
        let underlying_mint = ctx.create_mint()
            .pubkey(Keypair::new().pubkey())
            .decimals(6)
            .mint_authority(admin.pubkey())
            .create()
            .unwrap();
        eprintln!("[SETUP] Underlying mint (USDC, base): {}", underlying_mint);

        // Deposit token: SOL-like (9 decimals, will be priced at $298).
        let sol_mint = ctx.create_mint()
            .pubkey(Keypair::new().pubkey())
            .decimals(9)
            .mint_authority(admin.pubkey())
            .create()
            .unwrap();
        eprintln!("[SETUP] Deposit mint (SOL-like, 9 decimals): {}", sol_mint);

        // ================================================================
        // STEP 5: Create Squads ProgramConfig via account patching
        // ================================================================
        // On mainnet, Squads team initialized this once. In our test env it doesn't exist.
        // We create it manually with the right discriminator + data at the correct PDA.
        // The Squads program reads this during create_smart_account to determine the fee
        // and the next smart_account_index (used to derive the Settings PDA).
        let (squads_program_config, _) = Pubkey::find_program_address(
            &[b"smart_account", b"program_config"],
            &squads_program_id,
```

... continued RS

```rs
        );
        let squads_treasury = Keypair::new();

        ctx.create_account()
            .pubkey(squads_treasury.pubkey())
            .lamports(1_000_000_000)
            .owner(system_program::ID)
            .create()
            .unwrap();

        // Manually serialize ProgramConfig at the PDA address.
        // Layout: discriminator(8) + smart_account_index(16, u128 LE) + authority(32) +
  creation_fee(8, u64) + treasury(32) + reserved(64)
        {
            let config_size = 8 + 16 + 32 + 8 + 32 + 64; // 160 bytes
            let mut data = vec![0u8; config_size];
            data[..8].copy_from_slice(&SQUADS_PROGRAM_CONFIG_DISCRIMINATOR);
            // smart_account_index = 0 → first vault will use index 1 for Settings PDA
            // authority = admin (offset 24..56)
            data[24..56].copy_from_slice(admin.pubkey().as_ref());
            // creation_fee = 0 lamports (offset 56..64)
            data[56..64].copy_from_slice(&0u64.to_le_bytes());
            // treasury = where fees go (offset 64..96)
            data[64..96].copy_from_slice(squads_treasury.pubkey().as_ref());

            ctx.create_account()
                .pubkey(squads_program_config)
                .lamports(10_000_000)
                .owner(squads_program_id) // must be owned by Squads for Anchor validation
                .data(&data)
                .create()
                .unwrap();
        }
        eprintln!("[SETUP] Squads ProgramConfig PDA: {}", squads_program_config);

        // ================================================================
        // STEP 6: Derive ALL PDAs needed by initialize_vault
        // ================================================================

        // --- Exponent Vault PDAs (derived from exponent_vaults program) ---

        // Vault PDA: the core state account. seed_id makes it unique per vault.
        let seed_id: [u8; 8] = [1, 0, 0, 0, 0, 0, 0, 0];
        let (vault, vault_bump) = Pubkey::find_program_address(
            &[VAULT_SEED, &seed_id],
            &program_id,
        );
        // LP Mint: Token-2022 mint for vault shares. Vault PDA = mint authority.
        let (mint_lp, _) = Pubkey::find_program_address(
            &[b"mint_lp", vault.as_ref()],
            &program_id,
        );
        // LP Escrow: holds LP tokens locked during withdrawal queue and governance votes.
```

... continued RS

```rs
        let (token_lp_escrow, _) = Pubkey::find_program_address(
            &[b"token_lp_escrow", vault.as_ref()],
            &program_id,
        );
        eprintln!("[SETUP] Vault PDA: {} (bump: {})", vault, vault_bump);
        eprintln!("[SETUP] LP Mint PDA: {}", mint_lp);
        eprintln!("[SETUP] LP Escrow PDA: {}", token_lp_escrow);

        // Fee treasury: receives protocol's cut of LP on withdrawal fills. Random keypair is
fine.
        let fee_treasury_keypair = Keypair::new();
        let fee_treasury = fee_treasury_keypair.pubkey();

        // --- Squads PDAs (derived from Squads program using auto-incrementing counters) ---

        // Settings PDA: Squads derives this internally during create_smart_account.
        // Seeds: ["smart_account", "settings", (program_config.smart_account_index + 1).to_le
_bytes()]
        // Our ProgramConfig has index=0, so first Settings uses seed=1.
        let settings_seed: u128 = 1;
        let (squads_settings, _) = Pubkey::find_program_address(
            &[b"smart_account", b"settings", &settings_seed.to_le_bytes()],
            &squads_program_id,
        );

        // Squads Vault: sub-account 0 of the smart account. This is where deposited tokens go
.
        // Seeds: ["smart_account", settings_key, "smart_account", 0u8]
        let (squads_vault, _) = Pubkey::find_program_address(
            &[b"smart_account", squads_settings.as_ref(), b"smart_account", &[0u8]],
            &squads_program_id,
        );
        eprintln!("[SETUP] Squads settings: {}", squads_settings);
        eprintln!("[SETUP] Squads vault: {}", squads_vault);

        // Unused by the program but required in the accounts struct.
        let address_lookup_table = Keypair::new().pubkey();
        let lp_dst = Keypair::new().pubkey(); // initial_lp_amount must be 0, so this is never
 used

        // --- Squads policy PDAs (6 accounts for 2 policies) ---
        // initialize_vault creates 2 Squads policies via CPI:
        //   Policy 1: fill_withdrawal — constrains manager to only call fill_withdrawal
        //   Policy 2: token_transfer — allows vault PDA to move tokens via Squads
        //
        // Each policy creates 3 Squads accounts:
        //   1. SettingsTransaction — PDA: ["smart_account", settings, "transaction", tx_index
]
        //   2. Proposal           — PDA: ["smart_account", settings, "transaction", tx_index,
 "proposal"]
        //   3. Policy             — PDA: ["smart_account", "policy", settings, policy_seed]
        //
        // Counter state (traced from Squads source):
```

... continued RS

```rs
        //   After create_smart_account: transaction_index=0, policy_seed=Some(0)
        //   Policy 1: tx_index=1, policy_seed=1 (next = 0+1, then stored as Some(1))
        //   Policy 2: tx_index=2, policy_seed=2 (next = 1+1, then stored as Some(2))
        let tx_index_1: u64 = 1;
        let tx_index_2: u64 = 2;

        let (squads_settings_transaction, _) = Pubkey::find_program_address(
            &[b"smart_account", squads_settings.as_ref(), b"transaction", &tx_index_1.to_le_by
tes()],
            &squads_program_id,
        );
        let (squads_settings_transaction_transfer, _) = Pubkey::find_program_address(
            &[b"smart_account", squads_settings.as_ref(), b"transaction", &tx_index_2.to_le_by
tes()],
            &squads_program_id,
        );

        let (squads_proposal, _) = Pubkey::find_program_address(
            &[b"smart_account", squads_settings.as_ref(), b"transaction", &tx_index_1.to_le_by
tes(), b"proposal"],
            &squads_program_id,
        );
        let (squads_proposal_transfer, _) = Pubkey::find_program_address(
            &[b"smart_account", squads_settings.as_ref(), b"transaction", &tx_index_2.to_le_by
tes(), b"proposal"],
            &squads_program_id,
        );

        let policy_seed_1: u64 = 1;
        let policy_seed_2: u64 = 2;
        let (squads_policy, _) = Pubkey::find_program_address(
            &[b"smart_account", b"policy", squads_settings.as_ref(), &policy_seed_1.to_le_byte
s()],
            &squads_program_id,
        );
        let (squads_policy_transfer, _) = Pubkey::find_program_address(
            &[b"smart_account", b"policy", squads_settings.as_ref(), &policy_seed_2.to_le_byte
s()],
            &squads_program_id,
        );

        // --- Token accounts ---

        // Squads vault's SOL token account: where user SOL deposits land.
        // Owned by squads_vault (multisig), so only vault PDA (via Squads policy) can move
funds.
        let squads_token_account = ctx.create_token_account()
            .pubkey(Keypair::new().pubkey())
            .mint(sol_mint)
            .token_owner(squads_vault)
            .create()
            .unwrap();
        eprintln!("[SETUP] Squads SOL token account: {}", squads_token_account);
```

... continued RS

```rs
        // Vault PDA's own SOL token account: intermediate hop during withdrawals.
        let vault_token_account = ctx.create_token_account()
            .pubkey(Keypair::new().pubkey())
            .mint(sol_mint)
            .token_owner(vault)
            .create()
            .unwrap();
        eprintln!("[SETUP] Vault SOL token account: {}", vault_token_account);

        // ================================================================
        // STEP 7: Call initialize_vault
        // ================================================================
        // This single instruction does everything:
        //   1. Creates ExponentStrategyVault PDA with config (roles, fees, limits)
        //   2. Creates LP mint (Token-2022, vault PDA = authority)
        //   3. Creates LP escrow token account (vault PDA owns it)
        //   4. CPI → Squads create_smart_account (creates settings + vault sub-account)
        //   5. CPI → Squads create fill_withdrawal policy (settings_tx + proposal + approve
    + policy)
        //   6. CPI → Squads create token_transfer policy (settings_tx + proposal + approve +
     policy)
        //
        // Total: ~10 nested CPIs, needs 1.4M compute units (default 200K is not enough).

        // Mock Pyth oracle for SOL at $298.
        // IMPORTANT: Must use the same program ID that exponent_vaults validates against:
        //   pyth_v2_cpi::ID = "rec5EKMGg6MxZYaMdyBfgwp4d5rB9T1VQH5pJv5LtFJ"
        // Crucible's default is different, so we override with .program_id().
        let pyth_program_id: Pubkey = "rec5EKMGg6MxZYaMdyBfgwp4d5rB9T1VQH5pJv5LtFJ".parse().un
    wrap();
        let mock_oracle = ctx.create_mock_pyth_oracle()
            .price(298_00000000) // $298 with 8 decimals (Pyth convention)
            .exponent(-8)
            .confidence(100_000) // $0.001 confidence band
            .program_id(pyth_program_id) // match exponent's pyth_v2_cpi::ID
            .build()
            .unwrap();
        eprintln!("[SETUP] Mock Pyth oracle (SOL $298): {}", mock_oracle);

        // Squads settings/vault/policy accounts are CREATED by the CPI inside
    initialize_vault.
        // They must NOT exist beforehand (Anchor's init constraint requires
    system-program-owned empty accounts).

        // Batch ComputeBudget + InitializeVault in one atomic transaction.
        // ComputeBudget must be first instruction in the transaction.
        {
            use solana_instruction::Instruction;
            let compute_budget_id: Pubkey = "ComputeBudget111111111111111111111111111111".pars
    e().unwrap();
            let mut data = vec![2u8]; // variant 2 = SetComputeUnitLimit
            data.extend_from_slice(&1_400_000u32.to_le_bytes());
```

... continued RS

```rust
            let ix = Instruction { program_id: compute_budget_id, accounts: vec![], data };
            ctx.raw_call(ix).add_transaction().unwrap(); // queued, not sent yet
        }

        // Queue initialize_vault as second instruction in same batch.
        // token_entries is empty — we'll add tokens via manage_vault_settings after init.
        // This avoids needing to register prices in ExponentPrices first.
        ctx.program(program_id)
            .call(instruction::InitializeVault {
                manager: admin.pubkey(),        // must be roles.manager[0]
                fee_treasury_lp_bps: 500,       // 5% of LP burned on withdrawal goes to
    treasury
                management_fee_bps: 20000,      // 2% annual management fee (basis = 1,000,000
    )
                reserves_share_bp: 1000,        // 10% of AUM must stay liquid (basis = 10,000
    )
                token_entries: vec![],           // empty — add via manage_vault_settings
    later
                max_lp_supply: u64::MAX,         // no cap on vault size
                seed_id,                         // unique vault identifier
                initial_lp_amount: 0,           // must be 0 (program enforces this)
                lp_decimals: 6,                 // LP token decimal places
                vault_type: exponent_vaults::types::VaultType::Generic,
                roles: exponent_vaults::types::VaultRoles {
                    manager: vec![admin.pubkey()],   // can fill withdrawals, collect fees
                    curator: vec![],                  // can manage policies (unused)
                    allocator: vec![],                // can update positions (unused)
                    sentinel: vec![admin.pubkey()],  // can emergency-pause the vault
                },
                proposal_vote_config: None,     // no governance (uses defaults)
            })
            .accounts(accounts::InitializeVault {
                payer: admin.pubkey(),
                vault,
                exponent_prices,
                mint: underlying_mint,
                mint_lp,
                token_lp_escrow,
                fee_treasury,
                lp_dst,
                token_program: spl_token::id(),
                system_program: system_program::ID,
                // ATA program: ATokenGPvbdGVxr1b2hvZbsiqW5xWH25efTNsLJA8knL (hardcoded bytes)
                associated_token_program: Pubkey::new_from_array([
                    0x8c, 0x97, 0x25, 0x8f, 0x4e, 0x24, 0x89, 0xf1,
                    0xbb, 0x3d, 0x10, 0x29, 0x14, 0x8e, 0x0d, 0x83,
                    0x0b, 0x5a, 0x13, 0x99, 0xda, 0xff, 0x10, 0x84,
                    0x04, 0x8e, 0x7b, 0xd8, 0xdb, 0xe9, 0xf8, 0x59,
                ]),
                address_lookup_table,
                squads_program: squads_program_id,
                squads_program_config,
                squads_treasury: squads_treasury.pubkey(),
```

... continued RS

```
                squads_settings,
                squads_vault,
                squads_policy,
                squads_settings_transaction,
                squads_proposal,
                squads_policy_transfer,
                squads_settings_transaction_transfer,
                squads_proposal_transfer,
            })
            .signers(&[&*admin])
            .add_transaction()
            .unwrap();

        // Send both instructions atomically. If either fails, both roll back.
        let result = ctx.send_batch();

        match &result {
            Ok(Some(outcome)) if outcome.is_success() => {
                eprintln!("[SETUP] InitializeVault SUCCESS");

                // Now that LP mint exists, create fee_treasury as an LP token account.
                // initialize_vault only stores the pubkey — doesn't create the token account.
                // ExecuteWithdrawalFromReserves requires it to be an initialized TokenAccount

                ctx.create_token_account()
                    .pubkey(fee_treasury)
                    .mint(mint_lp)
                    .token_owner(admin.pubkey())
                    .create()
                    .unwrap();
                eprintln!("[SETUP] Fee treasury LP token account created: {}", fee_treasury);
            }
            Ok(Some(outcome)) => {
                eprintln!("[SETUP] InitializeVault FAILED:");
                if let Some(code) = outcome.error_code() {
                    eprintln!("  Error code: {}", code);
                }
                for log in outcome.logs() { eprintln!("  {}", log); }
                panic!("Setup failed: InitializeVault");
            }
            Ok(None) => {
                panic!("Setup failed: InitializeVault - empty batch");
            }
            Err(e) => {
                eprintln!("[SETUP] InitializeVault SEND ERROR: {:?}", e);
                panic!("Setup failed: InitializeVault");
            }
        }

        // ================================================================
        // STEP 8A: Register price in ExponentPrices via manage_prices
        // ================================================================
        // Register SOL→USDC price using Pyth oracle at $298.
```

... continued RS

```rs
        // price_mint = SOL (what we're pricing), underlying_mint = USDC (base denomination).
        // The Pyth oracle mock provides the price feed.
        let price_interface_seed: [u8; 8] = [1, 0, 0, 0, 0, 0, 0, 0];
        let (price_interface_accounts_pda, _) = Pubkey::find_program_address(
            &[b"price_interface_accounts", exponent_prices.as_ref(), &price_interface_seed],
            &program_id,
        );

        let result = ctx.program(program_id)
            .call(instruction::ManagePrices {
                actions: vec![
                    exponent_vaults::types::UpdatePriceAction::AddPrice {
                        price_type: exponent_vaults::types::PriceType::Pyth,
                        price_mint: sol_mint,          // SOL — what we're pricing
                        underlying_mint,               // USDC — base denomination
                        interface_accounts: vec![mock_oracle], // Pyth oracle account
                        price_interface_accounts_seed: price_interface_seed,
                    },
                ],
            })
            .accounts(accounts::ManagePrices {
                manager: admin.pubkey(),
                exponent_prices,
                system_program: system_program::ID,
            })
            // remaining_accounts:
            //   [0] price_interface_accounts_pda (writable — created by manage_prices)
            //   [1] sol_mint (read-only — for price_mint decimal resolution)
            //   [2] underlying_mint (read-only — for underlying_mint decimal resolution)
            .remaining_accounts_metas(vec![
                AccountMeta::new(price_interface_accounts_pda, false),
                AccountMeta::new_readonly(sol_mint, false),
                AccountMeta::new_readonly(underlying_mint, false),
            ])
            .signers(&[&*admin])
            .send();

        match &result {
            Ok(outcome) if outcome.is_success() => {
                eprintln!("[SETUP] ManagePrices AddPrice SUCCESS");
            }
            Ok(outcome) => {
                eprintln!("[SETUP] ManagePrices AddPrice FAILED:");
                if let Some(code) = outcome.error_code() {
                    eprintln!("  Error code: {}", code);
                }
                for log in outcome.logs() { eprintln!("  {}", log); }
                panic!("Setup failed: ManagePrices AddPrice");
            }
            Err(e) => {
                eprintln!("[SETUP] ManagePrices AddPrice SEND ERROR: {:?}", e);
                panic!("Setup failed: ManagePrices AddPrice");
            }
```

... continued RS

```rs
        }

        // ============================================================
        // STEP 8B: Add token entry to vault via manage_vault_settings
        // ============================================================
        // The vault was initialized with empty token_entries. Now we add the underlying
        // mint so users can deposit. AddTokenEntry needs 2 remaining_accounts:
        //   [0] token_squads_account — Squads-owned token account for this mint
        //   [1] token_vault_account — vault PDA-owned token account for this mint
        // Price is referenced via PriceId::Simple { price_id: 1 } (the price we just
    registered).
        // Price ID = 1 because the allocator assigns the first price at index 1 (0 is
    reserved/sentinel).
        // Use wrapper variant — for empty vaults, the vault PDA must sign.
        // WrapperManageVaultSettings handles this internally via invoke_signed.
        let result = ctx.program(program_id)
            .call(instruction::WrapperManageVaultSettings {
                actions: vec![
                    exponent_vaults::types::VaultSettingsAction::AddTokenEntry {
                        0: exponent_vaults::types::TokenEntry {
                            mint: sol_mint,
                            price_id: exponent_vaults::types::PriceId::Simple { price_id: 1 },
                            token_squads_account: squads_token_account,
                            token_account_vault: vault_token_account,
                            force_deallocate_policy_ids: vec![],
                        },
                    },
                ],
            })
            .accounts(accounts::WrapperManageVaultSettings {
                manager: admin.pubkey(),
                vault,
                exponent_prices,
                system_program: system_program::ID,
            })
            // remaining_accounts for AddTokenEntry: [squads_token_account,
    vault_token_account]
            .remaining_accounts(vec![squads_token_account, vault_token_account])
            .signers(&[&*admin])
            .send();

        match &result {
            Ok(outcome) if outcome.is_success() => {
                eprintln!("[SETUP] ManageVaultSettings AddTokenEntry SUCCESS");
            }
            Ok(outcome) => {
                eprintln!("[SETUP] ManageVaultSettings AddTokenEntry FAILED:");
                if let Some(code) = outcome.error_code() {
                    eprintln!("  Error code: {}", code);
                }
                for log in outcome.logs() { eprintln!("  {}", log); }
                panic!("Setup failed: ManageVaultSettings AddTokenEntry");
            }
```

... continued RS

```rs
            Err(e) => {
                eprintln!("[SETUP] ManageVaultSettings SEND ERROR: {:?}", e);
                panic!("Setup failed: ManageVaultSettings AddTokenEntry");
            }
        }

        // ================================================================
        // STEP 8C: Push initial price via update_price
        // ================================================================
        // PriceType::One should already have price=1.0 set during AddPrice, but
        // update_price ensures last_updated_slot is fresh for staleness checks.
        // remaining_accounts: [price_interface_accounts_pda] at index 0
        // interface_accounts_index points into remaining_accounts where the PDA is.
        // update_price remaining_accounts layout:
        //   [0]: PriceInterfaceAccounts PDA (read-only — stores oracle account references)
        //   [1]: Pyth oracle account (read-only — actual price data)
        let update_price_remaining = vec![price_interface_accounts_pda, mock_oracle];

        // Price freshness flow:
        //   1. update_price sets ExponentPrices.last_updated_slot = current_slot
        //   2. deposit_liquidity checks last_updated_slot == current_slot
        //   3. So both must run in the SAME slot
        //   4. But update_price also has a dedup guard: skips if last_updated_slot == current
    _slot
        //   5. So we need: advance slot → refresh oracle → update_price → deposit (all same
     slot)
        ctx.advance_slots(1);
        ctx.update_pyth_price(&mock_oracle, 298_00000000, -8).unwrap();
        ctx.refresh_pyth_oracle(&mock_oracle).unwrap();

        let result = ctx.program(program_id)
            .call(instruction::UpdatePrice {
                updates: vec![
                    exponent_vaults::types::UpdatePriceInput {
                        price_id: 1,
                        interface_accounts_index: 0,
                    },
                ],
            })
            .accounts(accounts::UpdatePrice {
                exponent_prices,
            })
            .remaining_accounts(update_price_remaining.clone())
            .signers(&[&*admin])
            .send();

        match &result {
            Ok(outcome) if outcome.is_success() => {
                eprintln!("[SETUP] UpdatePrice SUCCESS (SOL $298 at current slot)");
            }
            Ok(outcome) => {
                eprintln!("[SETUP] UpdatePrice FAILED:");
                if let Some(code) = outcome.error_code() {
```

... continued RS

```rust
                eprintln!("  Error code: {}", code);
            }
            for log in outcome.logs() { eprintln!("  {}", log); }
            panic!("Setup failed: UpdatePrice");
        }
        Err(e) => {
            eprintln!("[SETUP] UpdatePrice SEND ERROR: {:?}", e);
            panic!("Setup failed: UpdatePrice");
        }
    }

    // deposit_liquidity must run in the SAME slot as update_price (price freshness check)
    // Do NOT advance_slots here.

    // ============================================================
    // STEP 8D: Initial deposit to seed the vault with AUM
    // ============================================================
    // Create a temporary depositor to make the first deposit.
    // This gives the vault non-zero AUM and LP supply so withdrawals can work.
    // We use the admin as depositor for simplicity.
    let admin_token_account = ctx.create_token_account()
        .pubkey(Keypair::new().pubkey())
        .mint(sol_mint)
        .token_owner(admin.pubkey())
        .amount(100_000_000_000) // 100 SOL (9 decimals) = $29,800 worth
        .create()
        .unwrap();

    let admin_lp_account = ctx.create_token_account()
        .pubkey(Keypair::new().pubkey())
        .mint(mint_lp)
        .token_owner(admin.pubkey())
        .create()
        .unwrap();

    // deposit_liquidity needs remaining_accounts for AUM recalculation.
    // For each token_entry, pass the token_squads_account.
    // We have 1 token entry, so 1 remaining account.
    let result = ctx.program(program_id)
        .call(instruction::DepositLiquidity {
            mint: sol_mint,
            token_amount_in: 10_000_000_000, // 10 SOL (9 decimals) = $2,980 worth
            min_lp_out: 0, // no slippage check for setup
        })
        .accounts(accounts::DepositLiquidity {
            depositor: admin.pubkey(),
            vault,
            exponent_prices,
            token_src: admin_token_account,
            entry_token_vault: squads_token_account,
            token_lp_dst: admin_lp_account,
            mint_lp,
```

... continued RS

```rust
                token_program: spl_token::id(),
                token_program_lp: spl_token::id(),
                event_authority: Pubkey::find_program_address(&[b"__event_authority"], &progra
m_id).0,
                program: program_id,
            })
            .remaining_accounts(vec![squads_token_account])
            .signers(&[&*admin])
            .send();

        match &result {
            Ok(outcome) if outcome.is_success() => {
                eprintln!("[SETUP] DepositLiquidity SUCCESS (seeded vault with 10 SOL =
~$2,980)");
            }
            Ok(outcome) => {
                eprintln!("[SETUP] DepositLiquidity FAILED:");
                if let Some(code) = outcome.error_code() {
                    eprintln!("  Error code: {}", code);
                }
                for log in outcome.logs() { eprintln!("  {}", log); }
                panic!("Setup failed: DepositLiquidity");
            }
            Err(e) => {
                eprintln!("[SETUP] DepositLiquidity SEND ERROR: {:?}", e);
                panic!("Setup failed: DepositLiquidity");
            }
        }

        // ================================================================
        // STEP 9: Create test users
        // ================================================================
        // 3 users, each with: SOL (tx fees), deposit tokens (1M USDC), empty LP account.
        // The fuzzer will randomly pick users to perform deposit/withdraw actions.
        let mut users = Vec::new();
        for i in 0..3 {
            let user_kp = Rc::new(Keypair::new());
            ctx.create_account()
                .pubkey(user_kp.pubkey())
                .lamports(10_000_000_000) // 10 SOL for transaction fees
                .owner(system_program::ID)
                .create()
                .unwrap();

            // Funded SOL token account (1000 SOL at 9 decimals = 1_000_000_000_000 lamports =
 $298K worth)
            let user_token = ctx.create_token_account()
                .pubkey(Keypair::new().pubkey())
                .mint(sol_mint)
                .token_owner(user_kp.pubkey())
                .amount(1_000_000_000_000) // 1000 SOL
                .create()
                .unwrap();
```

... continued RS

```rs
            // Empty LP account — receives vault shares when user deposits
            let user_lp = ctx.create_token_account()
                .pubkey(Keypair::new().pubkey())
                .mint(mint_lp)
                .token_owner(user_kp.pubkey())
                .create()
                .unwrap();

            let mut token_accounts = HashMap::new();
            token_accounts.insert(sol_mint, user_token);

            users.push(UserData {
                keypair: user_kp.clone(),
                token_accounts,
                lp_token_account: user_lp,
            });
            eprintln!("[SETUP] User {}: {}", i, user_kp.pubkey());
        }

        eprintln!("[SETUP] Complete: {} users created", users.len());

        Self {
            ctx,
            program_id,
            squads_program_id,
            admin,
            exponent_prices,
            vault,
            vault_bump,
            seed_id,
            mint_lp,
            token_lp_escrow,
            fee_treasury,
            squads_settings,
            squads_vault,
            squads_token_account,
            vault_token_account,
            squads_policy_transfer,
            mock_oracle,
            pyth_program_id,
            price_interface_accounts_pda,
            underlying_mint,
            sol_mint,
            admin_token_account,
            prev_rate_num: 0,
            prev_rate_den: 0,
            users,
        }
    }


    fn refresh_price(&mut self) {
```

... continued RS

```rs
        let _ = self.ctx.update_pyth_price(&self.mock_oracle, 298_00000000, -8);
        let _ = self.ctx.refresh_pyth_oracle(&self.mock_oracle);
        let _ = self.ctx.program(self.program_id)
            .call(instruction::UpdatePrice {
                updates: vec![
                    exponent_vaults::types::UpdatePriceInput {
                        price_id: 1,
                        interface_accounts_index: 0,
                    },
                ],
            })
            .accounts(accounts::UpdatePrice {
                exponent_prices: self.exponent_prices,
            })
            .remaining_accounts(vec![self.price_interface_accounts_pda, self.mock_oracle])
            .signers(&[&*self.admin])
            .send();
    }

    // SVM-based action needed for coverage feedback — without it the fuzzer has no edges
    pub fn action_deposit(
        &mut self,
        #[range(0..3)] user_idx: usize,
        #[range(1..100_000_000_000)] amount: u64,
    ) {
        let user_idx = user_idx % self.users.len();
        let user_token = match self.users[user_idx].token_accounts.get(&self.sol_mint) {
            Some(acc) => *acc,
            None => return,
        };
        let balance = self.ctx.token_balance(&user_token);
        let amount = amount.min(balance);
        if amount == 0 { return; }
        self.refresh_price();
        let user_keypair = self.users[user_idx].keypair.clone();
        let user_lp = self.users[user_idx].lp_token_account;
        let _ = self.ctx.program(self.program_id)
            .call(instruction::DepositLiquidity {
                mint: self.sol_mint,
                token_amount_in: amount,
                min_lp_out: 0,
            })
            .accounts(accounts::DepositLiquidity {
                depositor: user_keypair.pubkey(),
                vault: self.vault,
                exponent_prices: self.exponent_prices,
                token_src: user_token,
                entry_token_vault: self.squads_token_account,
                token_lp_dst: user_lp,
                mint_lp: self.mint_lp,
                token_program: spl_token::id(),
                token_program_lp: spl_token::id(),
                event_authority: Pubkey::find_program_address(&[b"__event_authority"], &self.p
```

... continued RS

```
rogram_id).0,
                program: self.program_id,
            })
            .remaining_accounts(vec![self.squads_token_account])
            .signers(&[&*user_keypair])
            .send();
    }

    // ============================================================================
    // ACTION: Test Kamino Q68.60 → Number conversion precision
    // ============================================================================
    // Kamino stores market values as Q68.60 fixed-point (u128).
    // Exponent converts to Number (base-10, 1e12 precision).
    // This tests whether the conversion preserves value accurately.
    pub fn action_test_kamino_decode(
        &mut self,
        #[range(0..u64::MAX)] value_high: u64,
        #[range(0..u64::MAX)] value_low: u64,
    ) {
        let value_sf: u128 = (value_high as u128) << 64 | (value_low as u128);
        if value_sf == 0 { return; }

        const FRAC_BITS: u32 = 60;
        const DENOM: u128 = 1_000_000_000_000; // Number::DENOM = 1e12

        // ---- REPLICATE decode_fixed_point_to_number EXACTLY ----

        // Line 385: int_part = value_sf >> 60
        let int_part = value_sf >> FRAC_BITS;

        // Line 387-388: fractional_mask = 2^60 - 1
        let fractional_mask = (1u128 << FRAC_BITS) - 1;

        // Line 390: fractional_bits = value_sf & mask
        let fractional_bits = value_sf & fractional_mask;

        // Line 394-396: scaled_fraction = fractional_bits * DENOM
        let scaled_fraction = match fractional_bits.checked_mul(DENOM) {
            Some(s) => s,
            None => {
                fuzz_assert!(false, "OVERFLOW: frac_bits={} × DENOM overflows", fractional_bit
s);
                return;
            }
        };

        // Line 398: fractional_numerator = scaled_fraction >> 60
        let fractional_numerator = scaled_fraction >> FRAC_BITS;

        // Line 400-402: Number::from(int_part).checked_add(&Number::from_ratio(frac_num,
DENOM))
        //
        // Number::from(int_part) internally does: int_part * DENOM
```

... continued RS

```rust
        // Number::from_ratio(num, den) internally does: PreciseNumber::new(num) /
PreciseNumber::new(den)
        //   = (num * DENOM) / (den * DENOM) * DENOM = num * DENOM / den
        //   = fractional_numerator * DENOM / DENOM = fractional_numerator
        //
        // So the final internal representation (scaled by DENOM) is:
        //   decoded_raw = int_part * DENOM + from_ratio(fractional_numerator, DENOM)
        //
        // from_ratio(n, d) = (n * DENOM) / (d) where PreciseNumber division truncates
        // Since d = DENOM: from_ratio(frac_num, DENOM) = (frac_num * DENOM) / DENOM =
frac_num
        // BUT: PreciseNumber division may not be exact — let's replicate it precisely:
        //   PreciseNumber::new(frac_num) = frac_num * DENOM (as U256)
        //   PreciseNumber::new(DENOM) = DENOM * DENOM (as U256)
        //   division = (frac_num * DENOM) / (DENOM * DENOM) * DENOM
        //            = frac_num * DENOM * DENOM / (DENOM * DENOM)
        //            = frac_num  (exact, no remainder)
        //
        // So from_ratio(frac_num, DENOM) raw value = frac_num (in DENOM-scaled space)
        // And the full decoded raw = int_part * DENOM + frac_num

        let decoded_raw = match (int_part.checked_mul(DENOM))
            .and_then(|v| v.checked_add(fractional_numerator))
        {
            Some(v) => v,
            None => return, // overflow — very large int_part, skip
        };

        // ---- CHECK 1: integer part must match ----
        let expected_integer = value_sf >> FRAC_BITS;
        fuzz_assert!(
            int_part == expected_integer,
            "INTEGER MISMATCH: expected={} got={} value_sf={}",
            expected_integer, int_part, value_sf
        );

        // ---- CHECK 2: decoded value must not EXCEED truth ----
        // True value in DENOM-scaled space = value_sf * DENOM / 2^60
        // If decoded_raw > true_scaled, AUM is overstated.
        let true_scaled = match value_sf.checked_mul(DENOM) {
            Some(ts) => ts >> FRAC_BITS,
            None => return, // overflow for very large values, skip
        };

        fuzz_assert!(
            decoded_raw <= true_scaled + 1,
            "OVERSTATEMENT: decoded_raw={} > true_scaled={} +1 | value_sf={} int={} frac_num={
}",
            decoded_raw, true_scaled, value_sf, int_part, fractional_numerator
        );

        // ---- CHECK 3: precision loss bounded ----
        // The gap between decoded and truth should be at most 1 unit in DENOM scale.
```

... continued RS

```rust
        // true_scaled - decoded_raw < 2 (allows for truncation in >> 60)
        let gap = true_scaled.saturating_sub(decoded_raw);
        fuzz_assert!(
            gap <= 1,
            "PRECISION GAP: true={} decoded={} gap={} | value_sf={} frac_bits={} frac_num={}",
            true_scaled, decoded_raw, gap, value_sf, fractional_bits, fractional_numerator
        );
    }

    // ========================================================================
    // ACTION: Test fill withdrawal zero-token DoS
    // ========================================================================
    // Can a fill produce non-zero base_amount but zero token_amount?
    // If yes: user's LP is burned/consumed but they receive 0 tokens — DoS.
    // The program reverts with AmountTooSmall, but that means the withdrawal
    // is permanently stuck — the user can never complete it.
    pub fn action_test_fill_zero_token(
        &mut self,
        #[range(1_000_000..1_000_000_000)] lp_remaining: u64,    // 1 LP to 1000 LP (6
  decimals, so $1+ positions)
        #[range(1..10_000)] percent_bp: u64,
        #[range(100..5_000)] fee_bps: u64,
        #[range(1_000_000_000..10_000_000_000_000)] total_aum: u64,  // $1K to $10M
        #[range(1_000_000..10_000_000_000)] lp_supply: u64,          // 1 LP to 10K LP
        #[range(1_00000000..100_000_00000000)] price: u64,           // $1 to $100K
    ) {
        // Step 1: ceiling LP
        let numerator = match lp_remaining.checked_mul(percent_bp) {
            Some(n) => n, None => return,
        };
        let lp_amount = (numerator + 9_999) / 10_000;
        if lp_amount == 0 || lp_amount > lp_remaining { return; }
        if lp_supply == 0 || price == 0 { return; }

        // Step 2: floor fee
        let fee_cut = lp_amount * fee_bps / 10_000;
        let user_lp = lp_amount - fee_cut;
        if user_lp == 0 { return; }

        // Step 3: floor base (base_out_from_lp)
        let base_amount = ((total_aum as u128) * (user_lp as u128)
            / (lp_supply as u128)) as u64;
        if base_amount == 0 { return; }

        // Step 4: floor token (base_amount_to_token_amount)
        // For SOL(9dec)/USDC(6dec) with Pyth 8-decimal price:
        //   token = base × 10^(9-6) / (price / 10^8) = base × 10^11 / price
        let token_amount = ((base_amount as u128) * 100_000_000_000 / (price as u128)) as u64;

        // INVARIANT: non-zero base must produce non-zero tokens
        // If this fails: the fill reverts, user's withdrawal is stuck
        fuzz_assert!(
            token_amount > 0,
```

... continued RS

```
            "ZERO TOKEN DOS: base={} price={} → tokens=0 | lp={} user_lp={} fee={} aum={}
supply={}",
            base_amount, price, lp_amount, user_lp, fee_cut, total_aum, lp_supply
        );
    }

    // ========================================================================
    // ACTION: Test f64 precision in orderbook AUM calculation
    // ========================================================================
    // get_sy_in_offer uses f64 for AUM-critical math. f64 has 53-bit mantissa
    // (~15 digits). Large amounts or prices near 1.0 can cause meaningful divergence
    // from exact integer math. Tests both BuyYt and SellYt conversion paths.
    pub fn action_test_f64_precision(
        &mut self,
        #[range(1..u64::MAX)] amount: u64,
        #[range(100_000_000_000..1_200_000_000_000)] pt_price: u64,  // 0.1 to 1.2 in Number
        #[range(800_000_000_000..2_000_000_000_000)] sy_rate: u64,   // 0.8 to 2.0 in Number
    ) {
        let pt_f64 = pt_price as f64 / 1e12;
        let sy_f64 = sy_rate as f64 / 1e12;

        // ---- PATH 1: Virtual BuyYt ----
        // What the code does (divide first, then multiply):
        let ratio = pt_f64 / sy_f64;
        let divide_first = ((amount as f64) * ratio).floor() as u64;

        // More precise (multiply first, then divide):
        let multiply_first = ((amount as f64) * pt_f64 / sy_f64).floor() as u64;

        let diff = divide_first.abs_diff(multiply_first);

        fuzz_assert!(
            diff == 0,
            "BUYYT DIV-BEFORE-MUL: diff={} divide_first={} multiply_first={} | amount={} pt={:
.6} sy={:.6}",
            diff, divide_first, multiply_first, amount, pt_f64, sy_f64
        );

        // ---- PATH 2: Non-virtual SellYt ----
        // What the code does (divide first):
        let yt_ratio = (1.0 - pt_f64) / sy_f64;
        if yt_ratio > 0.0 {
            let yt_divide_first = ((amount as f64) * yt_ratio).floor() as u64;

            // Multiply first:
            let yt_multiply_first = ((amount as f64) * (1.0 - pt_f64) / sy_f64).floor() as u64
;

            let diff_yt = yt_divide_first.abs_diff(yt_multiply_first);

            fuzz_assert!(
                diff_yt == 0,
                "SELLYT DIV-BEFORE-MUL: diff={} divide_first={} multiply_first={} | amount={}
```

... continued RS

```
  pt={:.6} sy={:.6}",
                diff_yt, yt_divide_first, yt_multiply_first, amount, pt_f64, sy_f64
            );
        }
    }

    // ================================================================
    // ACTION: Test catastrophic cancellation in project_staged_sy
    // ================================================================
    // project_staged_sy computes: (1/old_rate - 1/new_rate) × staked_yt
    // When old and new rates are close, the subtraction cancels most significant
    // digits. The restructured form: staked_yt × (new - old) / (old × new)
    // avoids this. If results differ, cancellation lost meaningful precision.
    pub fn action_test_cancellation(
        &mut self,
        #[range(1..10_000_000_000_000_000_000)] staked_yt: u64,  // up to 10M tokens at 12
  decimals
        #[range(800_000_000_000..2_000_000_000_000)] old_rate: u64,
        #[range(1..1_000_000)] rate_bump: u64,
    ) {
        let new_rate = old_rate + rate_bump;
        let old_f64 = old_rate as f64 / 1e12;
        let new_f64 = new_rate as f64 / 1e12;
        if old_f64 == 0.0 || new_f64 == 0.0 { return; }

        // What the code does (cancellation path):
        let rate_delta = 1.0 / old_f64 - 1.0 / new_f64;
        let cancel_result = (rate_delta * staked_yt as f64).floor() as u64;

        // Restructured (no cancellation):
        let safe_result = (staked_yt as f64 * (new_f64 - old_f64) / (old_f64 * new_f64)).floor
  () as u64;

        let diff = cancel_result.abs_diff(safe_result);

        fuzz_assert!(
            diff == 0,
            "CANCELLATION: diff={} cancel={} safe={} | yt={} old={:.12} new={:.12}",
            diff, cancel_result, safe_result, staked_yt, old_f64, new_f64
        );
    }
}


#[invariant_test]
fn invariant_test(fixture: &mut ExponentFuzz) {
    let total_assets = fixture.ctx.token_balance(&fixture.squads_token_account) as u128;

    let total_supply = fixture.ctx.get_account(&fixture.mint_lp)
        .map(|acc| {
            if acc.data.len() >= 44 {
                u64::from_le_bytes(acc.data[36..44].try_into().unwrap()) as u128
            } else { 0u128 }
```

... continued RS

```
        })
        .unwrap_or(0u128);

    if total_supply == 0 || total_assets == 0 { return; }

    // Only check if we have a previous rate AND it wasn't reset by a price change
    if fixture.prev_rate_den > 0 {
        let current_cross = total_assets * fixture.prev_rate_den;
        let prev_cross = fixture.prev_rate_num * total_supply;

        fuzz_assert!(
            current_cross >= prev_cross,
            "EXCHANGE RATE DECREASED: was {}/{} now {}/{} (rate {:.10} -> {:.10})",
            fixture.prev_rate_num, fixture.prev_rate_den,
            total_assets, total_supply,
            fixture.prev_rate_num as f64 / fixture.prev_rate_den as f64,
            total_assets as f64 / total_supply as f64
        );
    }

    fixture.prev_rate_num = total_assets;
    fixture.prev_rate_den = total_supply;
}
```